



()
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Final Year Project Report

In partial fulfillment of the requirements for the State Engineer Degree in
Computer Science

Option: Intelligent Systems and Data (SD)

State of the Art on Transformer Pruning

Presented by:

MAHRAZ ABDELRAHMEN

REZIG HAMAZA

Supervised by:

MEZIANI Lila

GHORAB SARA

Host organization : LCSi ESI

Cohort: 2026/2027

Abstract

Transformer models have become standard tools in natural language processing, but their size makes deployment expensive. A dense model stores and evaluates every parameter, head, feed-forward neuron, and layer, even when some of this computation contributes little to the final prediction. Pruning tries to remove part of that computation while keeping the model usable. This report studies how pruning has been defined, scored, and applied to Transformer models.

The report starts from the basic formulation of pruning as a constrained compression problem, then reviews the main criteria used to decide what can be removed. It covers magnitude and activation-based methods, gradient and Taylor criteria, and second-order approaches such as Optimal Brain Damage and Optimal Brain Surgeon. It also separates unstructured sparsity from semi-structured and structured pruning, because these choices do not lead to the same hardware behavior. For Transformers, the discussion focuses on the units that can be removed in practice: weights, attention heads, feed-forward channels, and sometimes whole layers.

The review shows a recurring limitation. Many methods score units locally, then remove the lowest-ranked ones. This is simple and often effective at moderate compression, but it does not fully answer the harder question: how should a small remaining budget be distributed across the whole model? At high compression, the final allocation across layers can matter more than the score of any single unit. The report ends by framing this gap as the main motivation for search-based pruning methods.

Keywords: neural network pruning, Transformer compression, structured pruning, saliency criteria, search-based pruning

Résumé

Les modèles Transformer sont devenus des outils courants en traitement automatique du langage, mais leur taille rend leur déploiement coûteux. Un modèle dense stocke et exécute tous ses paramètres, toutes ses têtes d'attention, tous ses neurones feed-forward et toutes ses couches, même lorsqu'une partie de ce calcul contribue peu à la prédiction finale. L'élagage cherche à supprimer une partie de ce calcul tout en gardant un modèle exploitable. Ce rapport étudie la manière dont l'élagage a été formulé, évalué et appliqué aux modèles Transformer.

Le rapport commence par la formulation de l'élagage comme un problème de compression sous contrainte, puis passe en revue les principaux critères utilisés pour décider ce qui peut être retiré. Il présente les méthodes fondées sur la magnitude et les activations, les critères par gradient et développement de Taylor, ainsi que les approches du second ordre comme Optimal Brain Damage et Optimal Brain Surgeon. Il distingue aussi la parcimonie non structurée, semi-structurée et structurée, car ces choix n'ont pas les mêmes effets sur le matériel. Pour les Transformers, l'analyse se concentre sur les unités que l'on peut vraiment supprimer : les poids, les têtes d'attention, les canaux feed-forward et parfois des couches entières.

La revue fait apparaître une limite récurrente. Beaucoup de méthodes attribuent un score local aux unités, puis suppriment celles qui semblent les moins importantes. Cette stratégie reste utile à compression modérée, mais elle répond mal à une question plus difficile : comment répartir un budget très réduit sur l'ensemble du modèle ? À forte compression, la répartition finale entre les couches peut compter davantage que le score isolé d'une seule unité. Le rapport termine donc en présentant cette limite comme la principale motivation des méthodes d'élagage fondées sur la recherche.

Mots-clés : élagage de réseaux de neurones, compression de Transformers, élagage structuré, critères de saillance, élagage par recherche

Arabic Summary

Dedication

Acknowledgements

Contents

List of Figures

List of Tables

List of Acronyms and Abbreviations

Adam	Adaptive Moment Estimation
AE	Autoencoder
BERT	Bidirectional Encoder Representations from Transformers
BF16	Brain Float 16
CE	Cross-Entropy Loss
CNN	Convolutional Neural Network
CoFi	Coarse-to-Fine Structured Pruning via Learned Masks
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DarwinLM	Training-Aware Evolutionary Structured Pruning
ELU	Exponential Linear Unit
EMA	Exponential Moving Average
EvoPress	Evolutionary Search over Layer-Wise Compression Profiles
FFN	Feed-Forward Network
FLOPs	Floating Point Operations
FP16	16-bit Floating Point
FP32	32-bit Floating Point
FT	Fine-Tuning
GA	Genetic Algorithm
GELU	Gaussian Error Linear Unit
GLUE	General Language Understanding Evaluation

List of Acronyms and Abbreviations

GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
INT4	4-bit Integer
INT8	8-bit Integer
KD	Knowledge Distillation
KL	Kullback-Leibler Divergence
LLaMA	Large Language Model Meta AI
LLM	Large Language Model
LDLQ	LDL-Transpose Quantization
LLM-Pruner	Dependency-Aware Gradient-Based Structured Pruning
LoRA	Low-Rank Adaptation
MACs	Multiply-Accumulate Operations
MAE	Mean Absolute Error
MHA	Multi-Head Attention
MLP	Multi-Layer Perceptron
MNLI	Multi-Genre Natural Language Inference
MOP	Multi-Objective Optimization Problem
MRP	Multi-Weight Removal Process
MSE	Mean Squared Error
N:M	Semi-Structured Sparsity Pattern
NAS	Neural Architecture Search
NLP	Natural Language Processing
NSGA-II	Non-Dominated Sorting Genetic Algorithm II
OBS	Optimal Brain Surgeon
OPT	Open Pre-trained Transformer
PPL	Perplexity
QNLI	Question Natural Language Inference

List of Acronyms and Abbreviations

ReLU	Rectified Linear Unit
RMSNorm	Root Mean Square Normalization
RNN	Recurrent Neural Network
SGD	Stochastic Gradient Descent
SparseGPT	Second-Order Unstructured Pruning via Hessian Compensation
SQuAD	Stanford Question Answering Dataset
SVD	Singular Value Decomposition
Tanh	Hyperbolic Tangent
Wanda	Weight and Activation-Based Pruning Criterion
ZipLM	Runtime-Aware Second-Order Structured Pruning

General Introduction

The rapid progression of deep learning over the past decade has positioned large-scale neural networks as the dominant approach for a wide range of artificial intelligence tasks, from natural language understanding to code generation and complex reasoning. At the center of this progression stands the Transformer architecture, whose ability to model long-range dependencies has driven sustained improvements in performance across benchmarks. The models that result from training at this scale are, however, extremely costly to deploy. Their memory footprint, inference latency, and energy consumption exceed the constraints of most production environments, and make operation on resource-limited devices impractical without significant reduction of the model before use.

Model compression has emerged as the primary response to this gap between training-time capability and deployment-time feasibility. Several complementary directions have been developed: reducing the numerical precision of stored parameters, constraining the degrees of freedom used during adaptation, and removing parameters or computational units from the network entirely. Each direction can reduce the resource cost of running a model, and research in each area has produced measurable gains in efficiency with limited loss in accuracy.

Despite this progress, a fundamental difficulty remains. Most compression approaches arrive at their decisions by evaluating each removable unit independently, scoring it according to some local criterion before the effect of the full set of changes on the compressed model is known. This locality is a practical convenience but not a reliable basis for high-ratio compression: a unit that appears unimportant in isolation may carry significant weight once the units around it have also been removed, and the way the compression budget is allocated across the network has a large effect on the final model quality that no local score can anticipate.

The objective of this work is to address this allocation problem directly. Rather than scoring units one at a time and aggregating the results, the proposed framework for compression treats the configuration of the full network as the object to be optimized, searching over complete allocation profiles under a fixed computational budget. The goal is to find configurations that perform well as a whole, evaluated before costly recovery, and to do so in a way that preserves diversity across structurally different candidates and scales to the size of modern Transformer models.

This report is divided into two parts. The first part covers the state of the art and is composed of three chapters. Chapter 1 establishes the conceptual foundations required to follow the rest of the work, covering neural network architectures, training and optimization, the Transformer model, hardware efficiency metrics, and the evolutionary and multi-objective optimization meth-

ods the contribution draws on. Chapter 2 surveys model compression, presenting the formal setup of the principal compression families and the structural distinctions between them. Chapter 3 reviews the published literature within each compression family, examines the empirical evidence for their behavior, and analyzes the allocation limitation that motivates the proposed framework for compression.

The second part presents the contribution and is composed of two chapters. Chapter 4 describes the proposed framework for compression in full: the parameterization of the structural units subject to search, the budget model, the surrogate evaluation metric, the search algorithm, and the recovery procedure applied to the selected architecture. Chapter 5 reports the experimental results and compares the framework against relevant baselines under controlled compression budgets.

Chapter 1

Foundations

1.1 Introduction

Pruning modifies a trained neural network by removing parameters or larger structural units while trying to preserve its predictive behaviour. To understand what can be removed, and what the consequences of that removal are, the report first needs to define the objects that pruning acts on: neurons, weights, activations, layers, attention heads, feed-forward channels, and Transformer blocks. These foundations are therefore not introduced as general background only, but as the vocabulary needed to formulate pruning precisely.

The chapter begins with the artificial neuron as an affine transformation followed by a non-linear activation. This matters for pruning because weights and biases are the smallest trainable quantities in the model, while activations determine how information propagates after those parameters have been changed. The discussion then builds toward Multi-Layer Perceptrons, loss functions, gradient-based optimization, and backpropagation, since pruning quality is ultimately measured by how much the loss or output behaviour changes after part of the network is removed **Goodfellow2016; Rumelhart1986**.

The chapter then moves to the architectures most relevant to the pruning literature, with particular emphasis on Transformers. Transformer pruning is usually expressed in terms of executable structures such as attention heads, feed-forward channels, and layers rather than isolated scalar weights. For this reason, the chapter reviews the Transformer block, the attention and FFN components, and the computational bottleneck that motivates pruning as a compression method **vaswani2017attention**.

Finally, the chapter introduces hardware-aware efficiency metrics and genetic search concepts. These are needed because pruning is not only a question of reducing the number of parameters; it is also a question of whether the resulting architecture is cheaper to store, move through memory, and execute on real hardware. The following chapters build directly on these definitions to formalize pruning, compare existing pruning methods, and motivate structured search over compressed Transformer variants.

1.2 The Neuron as a Linear Model

The artificial neuron is a computational unit that takes a vector of inputs, applies a linear transformation, and then passes the result through a non-linear activation function. This structure allows neural networks to model complex, non-linear relationships in data. The linear part of the neuron is often referred to as an **affine transformation**, which combines a weighted sum of the inputs with a bias term. The non-linear activation function is crucial for enabling the network to learn and represent complex patterns, as it allows the model to capture non-linearities in the data. Common activation functions include ReLU, sigmoid, and tanh, each with its own properties and use cases.

1.2.1 Mathematical Formulation

The artificial neuron, formally introduced by **McCulloch1943**, serves as the fundamental building block of neural networks. Mathematically, it performs an **affine transformation** on its inputs followed by a non-linear activation.

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b \quad (1.1)$$

Given an input $\mathbf{x} \in \mathbb{R}^n$, weights $\mathbf{w} \in \mathbb{R}^n$ scale the importance of each feature, while the bias $b \in \mathbb{R}$ acts as an offset. Figure ?? shows this affine unit before the activation is applied: the weighted inputs are summed with the bias to produce the pre-activation response z , which is later transformed into $a = \phi(z)$.

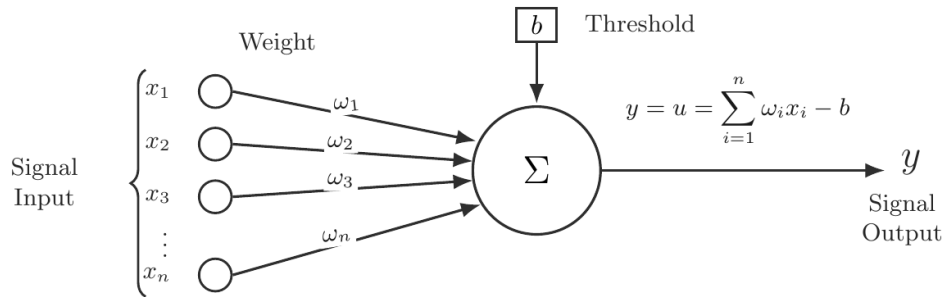


Figure 1.1: Affine unit of an artificial neuron. The input vector $\mathbf{x}$ is scaled by the weight vector $\mathbf{w}$, shifted by the bias b , and converted into the pre-activation response z before a non-linear activation is applied.

The term $\mathbf{w}^T \mathbf{x}$ measures how strongly the input matches the learned weights. This affine computation is the basic operation repeated across layers in larger neural networks.

Thus, the affine operation results in the neuron's raw response but in a way is still linear in its own right. This is why networks designed to model more complex problems pass this response through activation functions:

1.3 Activation Functions

The neuron introduced in the previous section computes an affine response $z = \mathbf{w}^T \mathbf{x} + b$. If neural networks only stacked affine transformations, the composition would collapse into a single affine transformation:

$$\mathbf{W}_2(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2 \mathbf{W}_1) \mathbf{x} + (\mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2). \quad (1.2)$$

Therefore, depth alone is not enough: neural networks require non-linear **activation functions** to model complex patterns **Goodfellow2016**.

1.3.1 Classical Saturating Activations

Sigmoid and hyperbolic tangent functions were often used in early networks. While these functions take real-valued inputs and output bounded values, which could be useful in the case of probabilities or normalized hidden states, they have very small derivatives when the input is very large or very small. This saturation causes learning to be very slow in deep networks **Glorot2010**; **Goodfellow2016**.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma : \mathbb{R} \rightarrow (0, 1). \quad (1.3)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \tanh : \mathbb{R} \rightarrow (-1, 1). \quad (1.4)$$

1.3.2 Rectified Activations

Rectified Linear Units (ReLU) became common in deep learning because they are simple, computationally cheap, and reduce saturation for positive inputs **Nair2010**. ReLU is defined as:

$$\text{ReLU}(x) = \max(0, x), \quad \text{ReLU} : \mathbb{R} \rightarrow [0, \infty). \quad (1.5)$$

Leaky ReLU keeps a small non-zero slope for negative inputs, reducing the risk that a unit becomes permanently inactive **Maas2013**:

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases} \quad \alpha \in (0, 1), \quad \text{LeakyReLU} : \mathbb{R} \rightarrow \mathbb{R}. \quad (1.6)$$

The Exponential Linear Unit (ELU) uses a smooth negative branch and a linear positive branch **Clevert2016**:

$$\text{ELU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha(e^x - 1), & x < 0, \end{cases} \quad \alpha > 0, \quad \text{ELU} : \mathbb{R} \rightarrow (-\alpha, \infty). \quad (1.7)$$

Modern Transformer architectures often use the Gaussian Error Linear Unit (GELU), a smooth activation that weights inputs by their value under the standard Gaussian cumulative distribution **Hendrycks2016**:

$$\text{GELU}(x) = x\Phi(x) \approx \frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right). \quad (1.8)$$

Figure ?? compares the shapes of these activation functions on the same input range. The plot makes the difference between saturating functions and rectified functions visible.

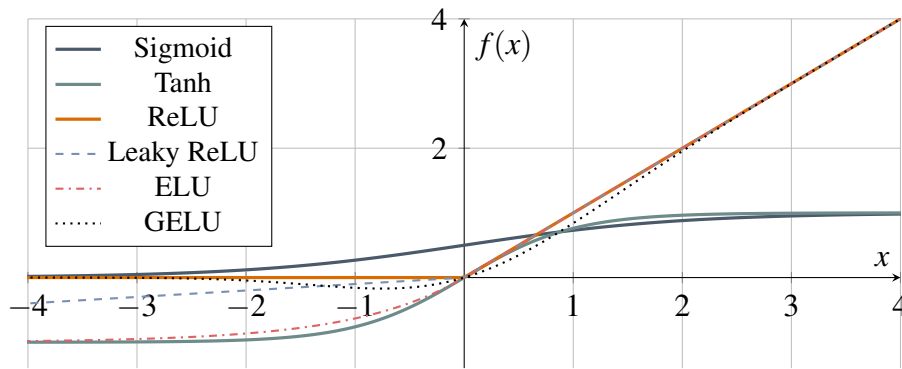


Figure 1.2: Comparison of common activation functions. Rectified Linear Unit (ReLU), Leaky ReLU, Exponential Linear Unit (ELU), and Gaussian Error Linear Unit (GELU) preserve a mostly linear positive branch, while sigmoid and hyperbolic tangent (tanh) saturate at both extremes.

In all of these cases the underlying idea remains largely the same since activation functions placed after affine transformations give deep composition its expressive power. From this observation, another question rises, which is how does this simple mechanism change when several neurons are combined and arranged into layers? This is the main discussion of the next point.

1.4 Building Neural Networks

The previous sections introduced a single neuron as an affine transformation followed by a non-linear activation. A single activated neuron can transform an input into a simple response, but practical models require many such units to work together. A Multi-Layer Perceptron extends the neuron model by organizing neurons into successive layers, so that each layer transforms the representation produced by the one before it.

1.4.1 The Multi-Layer Perceptron (MLP)

Going from a single neuron to an MLP increases expressiveness by composing affine transformations with non-linear activations across layers **Rumelhart1986**. MLPs are *feedforward* and typically *fully connected*, meaning each neuron connects to all neurons in the next layer.

Formally, if $\mathbf{h}^{(0)} = \mathbf{x}$ is the input, then each layer computes

$$\mathbf{h}^{(\ell)} = \phi\left(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \quad (1.9)$$

where $\mathbf{W}^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ are the parameters of layer ℓ , and ϕ is an activation function applied element-wise. The output of one layer becomes the input to the next, producing a sequence of learned transformations from raw input to prediction.

Diagram: Multi-Layer Perceptron Architecture

Figure ?? illustrates a typical MLP with one input layer, two hidden layers, and one output layer. The flow of information from input to output is called *forward propagation*: each layer applies a weighted transformation and activation, then passes its output to the next layer.

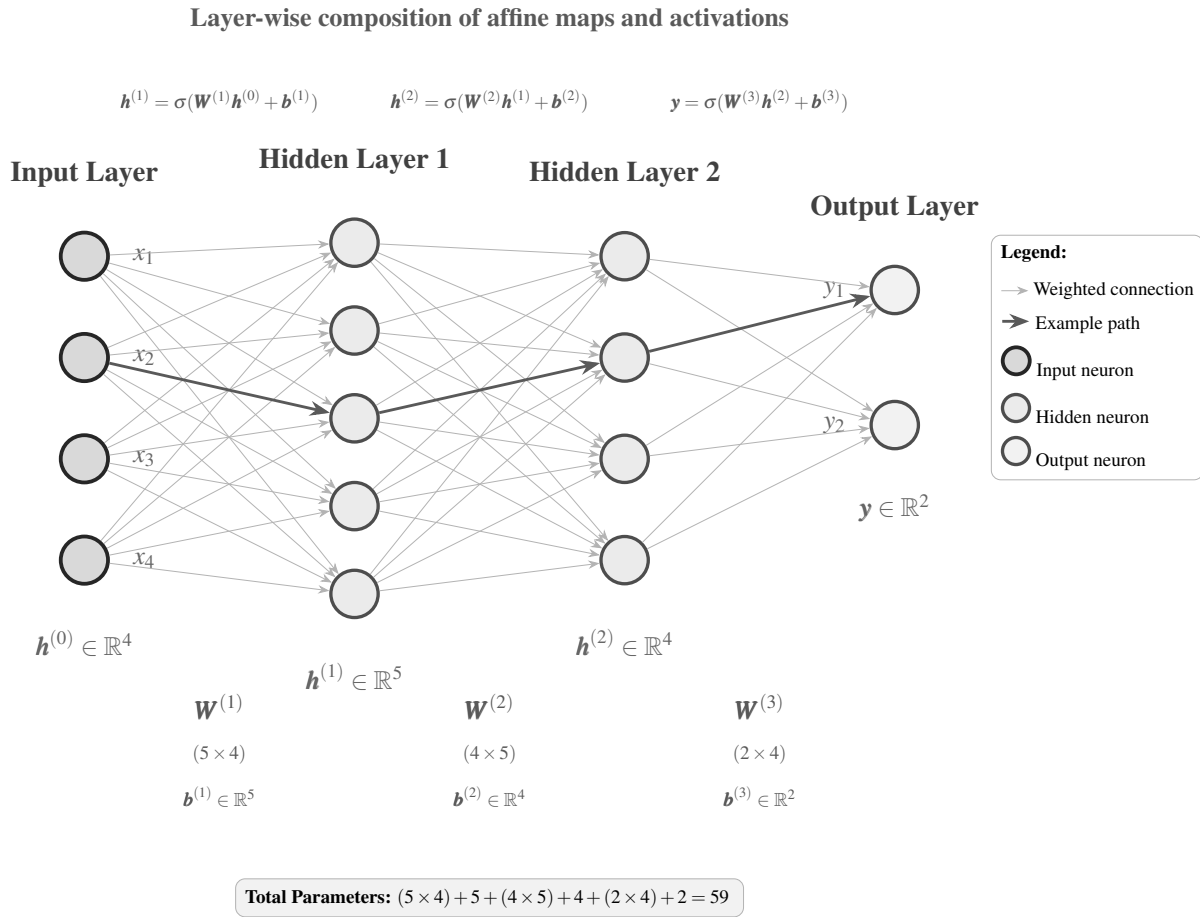


Figure 1.3: Fully connected Multi-Layer Perceptron (MLP) showing layer-wise information flow from input features to hidden representations and output predictions. The arrows represent weighted connections, and the highlighted path shows one possible route from an input feature to an output neuron.

The diagram shows the layer-wise transformations that define an MLP. The input layer stores the feature vector $\mathbf{h}^{(0)} = \mathbf{x}$. The first hidden layer computes $\mathbf{h}^{(1)}$ from all input features using

$\mathbf{W}^{(1)}$ and $\mathbf{b}^{(1)}$, then applies a non-linearity. The second hidden layer repeats the same operation on $\mathbf{h}^{(1)}$, producing $\mathbf{h}^{(2)}$. Finally, the output layer maps the last hidden representation to the prediction vector $\mathbf{y}$. The highlighted orange path illustrates one route by which information from a single input feature can influence an output neuron, and the pale connections show that every layer is fully connected to the next.

1.4.2 Hidden Representations

The hidden layers of an MLP are its internal states. The i -th hidden layer effectively performs the i -th transformation of the data, mapping it into the i -th coordinate space discovered through training. The earlier hidden layers perform combinations of input features, while the later layers perform combinations of the earlier layer outputs, creating higher-level representations related to the task in question. The output of the last hidden layer is typically transformed into the appropriate output space. For classification, this means class scores or probabilities. For regression, it means one or more continuous values.

The multi-layer representation is helpful, but it suffers from an obvious limitation. The MLP accepts its input as a single vector; therefore, there is no innate reason in the model for elements close to each other to be related, or for the elements of the vector to have an order. If such structure is required, it must be supplied by the architecture, which we will tackle later in Section ??.

For now, we remain with the MLP and ask a more basic question. Once the architecture has fixed how information can flow from inputs to outputs, how are the actual parameters of this flow chosen? This is the role of learning: the weights and biases must be adjusted from data so that the network output becomes closer to the desired target. The next section formulates this process as an optimization problem.

1.5 Learning as Optimization

In the above, we outlined how an MLP generates a prediction: by a composition of transformations applied at each layer. With forward computation as described, learning can be thought of as figuring out what the weights and biases should be given the data. Network construction then becomes an optimization problem: select the parameters that make the output as close as possible to the target on a training set.

Hence, a quantitative measure must be provided that can actually be minimized; this provides the rationale for empirical risk minimization, which is the topic of the next point.

1.5.1 Empirical Risk Minimization

Given a dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$, learning consists of finding parameters $\boldsymbol{\theta}$ that minimize the discrepancy between the model predictions $f_{\boldsymbol{\theta}}(\mathbf{x})$ and the target values. This is commonly

formulated as empirical risk minimization **Vapnik1998**:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \hat{R}(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}). \quad (1.10)$$

This empirical-risk objective in Equation (??) gives the general learning template: choose parameters by minimizing the average loss over the training set. It does not specify the loss itself. That choice is determined by the problem and the structure of the output, and it is the topic of the following subsection.

1.5.2 Loss Functions

Now we have an optimization problem. But what are we minimizing? A loss function accounts for the discrepancy between predictions and targets and makes explicit assumptions about the problem and the output space. We'll illustrate the three most common families, starting from classification to regression.

Cross-Entropy Loss

For classification tasks, neural networks typically produce a probability distribution over K classes via a softmax output. Given a predicted distribution $\hat{\mathbf{y}} \in \Delta^K$ and a target distribution $\mathbf{y}$ (often a one-hot vector), the cross-entropy loss is defined as:

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{y}_k. \quad (1.11)$$

The cross-entropy expression in Equation (??) quantifies the number of bits required on average to encode symbols from $\mathbf{y}$ using a code optimal for $\hat{\mathbf{y}}$. For a hard one-hot label, this becomes the negative log-likelihood of the true class $\mathcal{L}_{\text{CE}} = -\log \hat{y}_c$, where c is the index of the true class. This has the effect of pushing most of the probability mass onto the correct class, and preventing it from going anywhere else. For a hard label, this works out fine, but the moment the target becomes a distribution instead of a single class, the problem becomes more explicitly about how to quantify distance between distributions, which motivates the following point.

Kullback–Leibler Divergence

Cross-entropy implicitly compares two distributions, which raises a natural question: what is the right way to measure how far apart two probability distributions are? The Kullback–Leibler (KL) divergence **Kullback1951** provides a principled answer rooted in information theory. Rather than measuring total encoding cost, it isolates the *excess* cost incurred by using $\hat{\mathbf{y}}$ in place of the true distribution $\mathbf{y}$:

$$D_{\text{KL}}(\mathbf{y} \parallel \hat{\mathbf{y}}) = \sum_{k=1}^K y_k \log \frac{y_k}{\hat{y}_k}. \quad (1.12)$$

Expanding the KL divergence in Equation (??) reveals the exact relationship with cross-entropy:

$$D_{\text{KL}}(\mathbf{y} \parallel \hat{\mathbf{y}}) = \underbrace{-\sum_k y_k \log \hat{y}_k}_{\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y})} - \underbrace{\left(-\sum_k y_k \log y_k\right)}_{H(\mathbf{y})}, \quad (1.13)$$

where $H(\mathbf{y})$ is the entropy of the target distribution. The decomposition in Equation (??) shows why cross-entropy and KL divergence lead to the same optimization problem when the target distribution is fixed: $H(\mathbf{y})$ does not depend on the model parameters. If the target is a one-hot label then $H(\mathbf{y}) = 0$ and the two functions are equal; if the target is a soft distribution, then $H(\mathbf{y}) > 0$, and KL divergence is the clearly superior objective. The above loss functions are for when the output is a discrete probability distribution. For real-valued regression problems, an alternative family of loss functions are used.

Regression Losses

For regression, the output is a continuous value rather than a class index, and the appropriate loss changes accordingly. The mean squared error (MSE) is the standard choice:

$$\mathcal{L}_{\text{MSE}}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2. \quad (1.14)$$

The MSE objective in Equation (??) penalizes large regression errors quadratically. Under a Gaussian noise assumption on the targets, minimizing it is equivalent to maximum likelihood estimation of the model parameters. When the output distribution is heavier-tailed or contains outliers, the mean absolute error (L1 loss) $\mathcal{L}_{\text{MAE}} = \frac{1}{m} \sum_i |\hat{y}^{(i)} - y^{(i)}|$ is more robust, as it does not square large deviations.

With the loss function established, the remaining question is how the resulting error signal propagates back through the network to update the parameters, which is the subject of the next section.

1.6 Algorithms for Gradient-Based Optimization

Training neural networks is usually a problem of gradient-based optimization. The objective is to find a parameter vector $\theta \in \mathbb{R}^d$ that minimizes the scalar objective $J(\theta)$ defined above. In modern neural networks, the parameter space is high-dimensional and the objective is generally non-convex, so a closed-form solution for the optimal parameters is usually unavailable. Training therefore proceeds by iteratively updating the parameters in directions suggested by the gradient of the loss **Bottou2018**.

The gradient $\nabla J(\theta)$ gives the direction of steepest local increase of the objective. Optimization methods use this information in reverse: they adjust the parameters in a direction that is expected to reduce the loss. This idea gives the basic gradient descent algorithm and also underlies the adaptive optimizers used in current neural-network training.

1.6.1 Gradient Descent

Gradient descent updates parameters in the negative-gradient direction. For iteration t and learning rate $\eta > 0$, the update is:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t). \quad (1.15)$$

The learning rate controls the step size. If it is too small, optimization may progress slowly; if it is too large, the updates can overshoot useful regions of the objective surface. In large-scale learning, the exact gradient over the full dataset is often replaced by a stochastic estimate computed on a mini-batch. This gives stochastic gradient descent, which is cheaper per update and is widely used for neural-network training **Bottou2018**.

1.6.2 Adam

Adam is an adaptive gradient-based optimizer that combines momentum with per-parameter learning-rate adaptation **kingma2015**. Instead of using only the current gradient, Adam keeps exponential moving averages of the first and second moments of the gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (1.16)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (1.17)$$

where $g_t = \nabla J(\theta_t)$ is the gradient estimate at iteration t , m_t tracks the average direction of recent gradients, and v_t tracks their squared magnitude.

Because both moving averages are initialized at zero, Adam applies bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (1.18)$$

The parameter update is then:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (1.19)$$

Adam is commonly used because it is robust to gradient-scale differences across parameters and often requires less manual learning-rate tuning than plain gradient descent. Later variants such as AdamW decouple weight decay from the adaptive update, which is useful in many deep-learning settings **Loshchilov2019**.

Gradient descent and Adam describe how parameters should be updated, but they all still presuppose the necessary gradients being available. In deep, multi-layered networks calculating such gradients may be computationally expensive and hence demands a separate procedure called backpropagation, which we will tackle in the next point.

1.7 Backpropagation

Gradient descent and Adam require gradients of the loss with respect to the model parameters. Backpropagation computes these gradients efficiently by applying the chain rule from the output

layer back through the network **Rumelhart1986**. In compact form, it provides the quantities

$$\nabla_{\theta} \mathcal{L}, \tag{1.20}$$

where θ denotes the trainable parameters and $\mathcal{L}$ is the loss function.

For this pruning-focused report, the detailed derivation is not needed. What matters is that backpropagation links model error to individual parameters and intermediate structures. This link is used later by gradient-based pruning criteria, Taylor approximations, Fisher-based scores, and recovery stages after structural removal.

With forward propagation defining prediction and backpropagation defining gradient computation, the next step is to move from general multilayer networks to the architectures most relevant to pruning. The following section focuses on Transformer-based models.

1.8 Specialized Architectures

While Multi-Layer Perceptrons offer a generic learning model over vector inputs, they do not inherently leverage any underlying structure of the data. For this report, the architecture of interest is the Transformer, which has become the dominant model for sequence modeling tasks, particularly in natural language processing, and is the target of the compression methods discussed in later chapters.

1.8.1 Transformer Networks

Transformer networks, introduced by **vaswani2017attention**, changed sequence modeling by replacing recurrent processing with attention-based computation. Instead of processing tokens one after another, a Transformer can compare tokens across the whole sequence in parallel. This makes it highly effective for language modelling, translation, summarization, and other sequence tasks. The original architecture was an encoder-decoder model for machine translation, while later systems often use encoder-only variants such as BERT **devlin2019bert** or decoder-only variants such as LLaMA **touvron2023llama2**.

1.8.2 Core Architectural Components: Building Blocks of the Transformer

Before separating the model into encoder and decoder stacks, it is useful to define the components repeated inside those stacks. Transformer layers are built from token embeddings, positional information, attention projections, feed-forward channels, residual connections, and normalization layers. These components are also the main targets for compression, because they determine both parameter count and runtime cost.

Input Representation: Embeddings and Positional Encoding

Every token is mapped to a continuous embedding vector. Since self-attention does not inherently encode token order, positional information is added to the token representations. The original Transformer uses sinusoidal positional encodings **vaswani2017attention**:

$$PE_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right). \quad (1.21)$$

Self-Attention Mechanism

Self-attention allows each token representation to aggregate information from the rest of the sequence. The mechanism uses three learned projections of each input: queries, keys, and values **vaswani2017attention**. Queries represent what a token is looking for, keys represent what each token offers for comparison, and values contain the information that will be aggregated.

For an input sequence representation $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$, the projections are:

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V. \quad (1.22)$$

Scaled dot-product attention is then computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (1.23)$$

The factor $\sqrt{d_k}$ prevents dot products from becoming too large, which would push the softmax into saturated regions with weak gradients.

Multi-Head Attention

Multi-head attention applies several attention mechanisms in parallel, allowing the model to attend to different representation subspaces and token relationships at the same time **vaswani2017attention**. For h heads:

$$\text{head}_i = \text{Attention}(\mathbf{XW}_i^Q, \mathbf{XW}_i^K, \mathbf{XW}_i^V), \quad i = 1, \dots, h. \quad (1.24)$$

The head outputs are concatenated and projected back to the model dimension:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O. \quad (1.25)$$

Here, $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is the output projection. Attention heads and their projection matrices are important pruning targets because entire heads can sometimes be removed with limited accuracy loss.

Feed-Forward Network

After attention mixes information across tokens, a position-wise Feed-Forward Network (FFN) processes each token representation independently:

$$\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2. \quad (1.26)$$

The FFN usually expands the hidden representation to a larger intermediate dimension d_{ff} and then projects it back to d_{model} . In the original Transformer, d_{ff} is typically much larger than d_{model} , making FFN channels a major source of parameters and a natural target for structured pruning.

Residual Connections and Layer Normalization

Each attention or FFN sub-layer is wrapped by a residual path and layer normalization. Residual connections preserve a direct path for information and gradients, while normalization stabilizes the scale of hidden representations. This design is essential for training deep Transformer stacks and is also important when later compression methods remove heads, channels, or blocks [vaswani2017attention](#); [He2016](#); [Ba2016](#).

1.8.3 Encoder and Decoder Architecture

With the core components defined, the encoder and decoder can be described cleanly. The full encoder-decoder Transformer contains two main stacks:

- **Encoder:** Processes the complete input sequence in parallel and produces contextual source representations.
- **Decoder:** Generates the output sequence autoregressively while attending to both previous target tokens and the encoder output.

The encoder converts the source sequence into a contextual memory representation $\mathbf{H}_{\text{enc}}$. The decoder uses this memory through cross-attention while also using masked self-attention to prevent access to future target tokens. The complete architecture is shown in Figure ??.

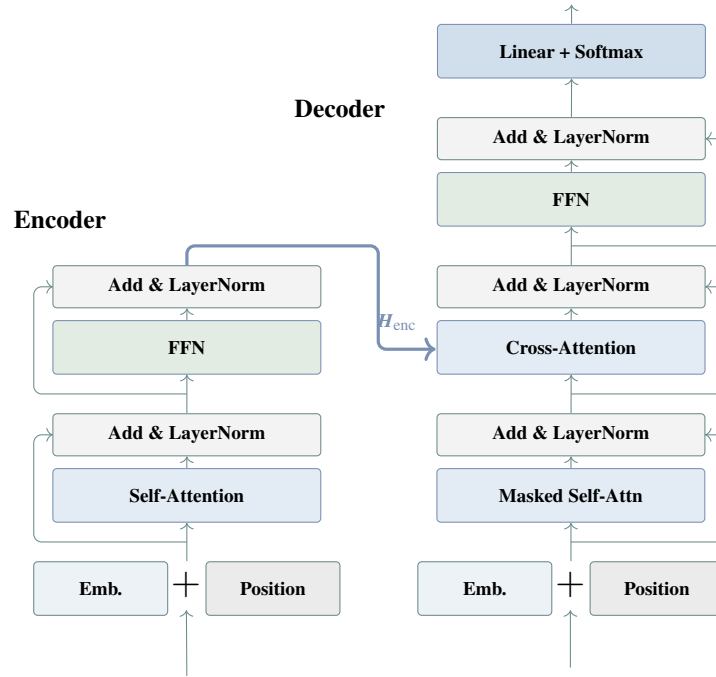


Figure 1.4: Complete Encoder-Decoder Transformer architecture. The encoder memory matrix H_{enc} (highlighted in blue) is fed to all decoder layers for cross-attention. Residual connections (curved arrows) enable stable gradient flow throughout the network.

The Encoder Stack

An encoder layer contains multi-head self-attention followed by a feed-forward network, with residual connections and normalization around each sub-layer. In encoder self-attention, the queries, keys, and values all come from the same source sequence. This makes the encoder bidirectional: every input token can attend to every other input token. Figure ?? shows the internal order of these operations before the resulting representation is passed to the decoder. After N encoder layers, the output is the memory matrix $H_{\text{enc}} \in \mathbb{R}^{n \times d_{\text{model}}}$, which summarizes the source sequence for the decoder.

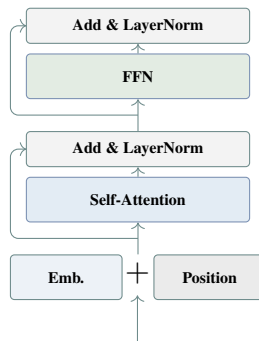


Figure 1.5: Transformer encoder layer architecture. The input embedding (Emb.) is combined with positional information, processed by self-attention and a Feed-Forward Network (FFN), and stabilized by residual connections with Layer Normalization (LayerNorm).

The Decoder Stack

A decoder layer adds masked self-attention and cross-attention before the feed-forward network, enabling autoregressive generation while attending to encoder outputs. Masked self-attention restricts token t to positions $\leq t$, preventing information leakage from future target tokens. Cross-attention then links the decoder to the encoder: decoder states act as queries, while encoder states provide keys and values. Figure ?? shows this additional cross-attention block, which is the main structural difference between the decoder layer and the encoder layer.

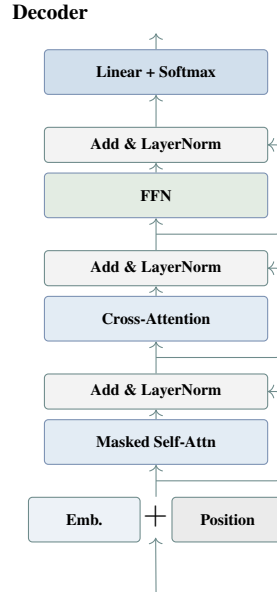


Figure 1.6: Transformer decoder layer architecture. The decoder contains masked self-attention for previous target tokens, cross-attention for reading encoder outputs, and a Feed-Forward Network (FFN), each wrapped with residual connections and Layer Normalization (LayerNorm).

Encoder-Decoder Information Flow

The complete sequence-to-sequence computation proceeds as follows:

1. **Encoding phase.** Source tokens $\mathbf{x} = (x_1, \dots, x_n)$ are embedded, combined with positional information, and passed through N encoder layers. The output is $\mathbf{H}_{\text{enc}}$.
2. **Decoding phase.** Target tokens $\mathbf{y} = (y_1, \dots, y_t)$ are embedded and processed autoregressively. Each decoder layer applies masked self-attention, cross-attention over $\mathbf{H}_{\text{enc}}$, and a feed-forward transformation.
3. **Output phase.** The final decoder representation is projected to vocabulary logits. A softmax layer converts logits into token probabilities, and the selected token is fed back into the decoder at the next step.

At this point in the report, we can view the Transformer as an arrangement of embedding layers, attention layers, feed-forward functions, and encoder–decoder connections. This architecture accounts for why it performs well on sequence modelling tasks, since each position can refine its representation using information from the rest of the sequence. This expressive structure, however, is computationally demanding. The next point examines where this cost arises inside the Transformer architecture.

1.8.4 The Computational Bottleneck of Transformers

While powerful, the Transformer’s success comes at a significant computational cost, which forms a central motivation for the compression techniques explored in this report. The standard Transformer architecture replaces recurrence with self-attention, enabling strong parallelism and high performance across sequence modelling tasks **vaswani2017attention**. However, this advantage comes with a major bottleneck: in full self-attention, each token attends to every other token in the sequence. For a sequence of length n , this requires an attention score matrix of size $n \times n$, making the attention component scale quadratically with sequence length in both computation and memory **beltagy2020longformer**; **tay2020efficient**.

This quadratic behaviour has direct practical implications. Doubling the sequence length from 512 to 1024 multiplies the attention matrix size by four, since $1024^2/512^2 = 4$. As a result, processing long documents or long-context inputs becomes substantially more expensive, which has motivated a large body of work on efficient Transformer variants and sparse or linear attention mechanisms **beltagy2020longformer**; **tay2020efficient**.

At the same time, modern language models benefit strongly from scaling model size, training data, and compute **kaplan2020scaling**. Later scaling-law work further showed that model size and the number of training tokens should be scaled together under a fixed compute budget, highlighting the continuing importance of large-scale training **hoffmann2022training**. This creates a tension between scale and deployment: larger models often achieve better performance, but practical inference is constrained by latency, memory, energy, and hardware availability. This tension motivates compression methods such as pruning, quantization, and knowledge distillation **han2015deep**; **sanh2019distilbert**.

The computational bottleneck defined above can be considered in several different ways. A Transformer may be expensive because of the size of its parameters, the memory required for activations, the number of arithmetic operations it performs, or the time required to run it on a specific hardware platform. Before discussing model efficiency in more detail, it is therefore useful to introduce the most common measures used to assess computational cost.

1.8.5 Hardware-Aware Efficiency Metrics

Compression is evaluated through several distinct efficiency metrics. Parameter count, memory footprint, arithmetic cost, latency, and throughput capture different aspects of model execution. Hardware-aware evaluation therefore requires separate treatment of storage cost, arithmetic cost, and runtime behavior **Sze2017EfficientDNN**.

Parameter Count and Model Size

Let $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^L$ denote the parameters of a network. The total parameter count is

$$P = \sum_{l=1}^L (|W^{(l)}| + |b^{(l)}|). \quad (1.27)$$

If the parameters are stored with precision b_w bits, the static model footprint is

$$M_{\text{model}} = \frac{P b_w}{8} \text{ bytes}. \quad (1.28)$$

This follows from counting the stored elements and multiplying by the number of bytes used by each element **PyTorchNumel**; **PyTorchElementSize**. This quantity determines whether a model fits within device memory and how much parameter traffic must be sustained during inference. Quantization reduces b_w , pruning reduces the number of active parameters, and low-rank factorization reduces the parameterization itself.

For sparse models, storage depends on both the remaining weights and the associated meta-data. If P_{nz} denotes the number of non-zero weights and M_{meta} the storage needed for indices or masks, then

$$M_{\text{sparse}} = \frac{P_{\text{nz}} b_w}{8} + M_{\text{meta}}. \quad (1.29)$$

MACs, FLOPs, and Arithmetic Intensity

For a dense matrix multiplication $Y = AB$ with $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, the arithmetic cost is

$$\text{MACs}(A, B) = mkn, \quad \text{FLOPs}(A, B) \approx 2mkn. \quad (1.30)$$

The distinction is conventional: one multiply-accumulate corresponds to one MAC, while FLOP counting often treats the multiply and the addition as two floating-point operations **NVIDIAMatrixMultiplication**. In Transformer layers, the feed-forward subnetwork contributes large dense matrix products, while self-attention adds the sequence-dependent $O(n^2 d_{\text{model}})$ cost of score computation and value aggregation.

Arithmetic cost alone is insufficient for hardware analysis. A useful complementary quantity is the arithmetic intensity

$$I = \frac{\text{FLOPs}}{\text{Bytes Moved}}, \quad (1.31)$$

which measures how much computation is performed per byte transferred **Williams2009Roofline**; **NVIDIAMatrixMultiplication**. Low arithmetic intensity indicates a memory-bound workload, whereas high arithmetic intensity indicates a compute-bound workload.

Bandwidth, Latency, and Throughput

Memory bandwidth measures the rate at which data can be transferred between memory and compute units. If B_{move} bytes are transferred during an operation that takes time T , the effective bandwidth demand is

$$\text{BW}_{\text{eff}} = \frac{B_{\text{move}}}{T}. \quad (1.32)$$

This follows the standard effective-bandwidth definition used for GPU kernels **CUDABestPractices**. When parameter streaming and activation movement dominate execution, bandwidth becomes the primary runtime constraint.

Latency and throughput characterize runtime from complementary viewpoints. Latency is the time required to complete one inference request or one decoding step **Sze2017EfficientDNN**:

$$T_{\text{latency}} = T_{\text{compute}} + T_{\text{memory}} + T_{\text{overhead}}. \quad (1.33)$$

Throughput measures completed work per unit time, for example requests per second or generated tokens per second **Sze2017EfficientDNN**:

$$\text{Throughput} = \frac{N_{\text{outputs}}}{T}. \quad (1.34)$$

In large autoregressive models, latency and throughput often follow different scaling trends. Batching can increase throughput while single-request latency remains high. Parameter compression can reduce memory transfer while leaving runtime gains limited when execution is poorly aligned with the hardware schedule.

The above metrics express ways in which candidate solutions can be measured in both the predictive and computational sense. The measurements derived in these senses are appropriate when a problem has numerous candidate solutions and direct enumeration of them all is infeasible. Hence, in the next section genetic algorithms are introduced as one form of a general algorithm which can be used to search very large solution spaces given various measures by which to evaluate them.

1.9 Genetic Algorithms and Search

With the hardware-aware metrics introduced above, it becomes possible to compare model variants concretely. In particular, different designs may be compared with regard to prediction accuracy, parameter count, memory footprint, computational cost, memory bandwidth usage,

and latency/throughput. In compression, these variants may correspond to different pruning masks, retained heads, feed-forward widths, removed layers, or layer-wise numerical precision.

Taken together, these choices form a large design space. Since enumerating all possible candidates is usually prohibitive, a search procedure is needed to explore this space and identify candidates that satisfy efficiency requirements while preserving prediction accuracy.

To manage this search, one can maintain a set of candidate designs instead of tracking a single design at a time. Each candidate corresponds to a possible compressed model. The candidates are evaluated, the more relevant ones are kept with higher probability, and new candidates are generated from existing ones through small changes or recombination. By repeating this process, the candidate set can move toward desirable regions of the design space.

A population-based search algorithm of this kind is known as a genetic algorithm. In this terminology, a candidate compressed model is an individual, the set of candidates is a population, and the evaluation measure is the fitness function. The following section introduces these notions together with encoding, selection, crossover, and mutation, then links them to compression design choices **holland1975adaptation**; **goldberg1989genetic**; **deb2001multi**.

1.9.1 Genetic Algorithm Fundamentals

A Genetic Algorithm (GA) is a population-based search method inspired by biological evolution. It maintains candidate solutions, represents them in a form that can be modified, evaluates their quality, and repeatedly generates new candidates through selection, crossover, and mutation **holland1975adaptation**; **goldberg1989genetic**; **michalewicz1996genetic**.

Components of a Genetic Algorithm

The basic object manipulated by a GA is an individual, also called a chromosome. Each individual encodes one candidate solution to the problem. A population is then a set of such individuals evaluated during the same generation. The fitness function assigns a quality score to each candidate, and the genetic operators define how new candidates are produced from existing ones **goldberg1989genetic**; **michalewicz1996genetic**. These definitions establish what the algorithm manipulates; the next issue is how a candidate is written as a chromosome.

In model compression, an individual may represent a compressed architecture. For example, consider the chromosome

$$\mathbf{c} = [1, 0, 1, 1 \mid 2048, 1024 \mid 8, 4, 4, 8], \quad (1.35)$$

where the first block indicates which attention heads are retained, the second block gives selected feed-forward widths, and the final block assigns layer-wise bit-widths. The fitness value then measures how good this candidate is with respect to the desired objective, such as validation accuracy, parameter count, memory footprint, or latency.

Encoding Methods for Individuals

The encoding determines how a candidate solution is written as a chromosome and therefore what kind of changes the algorithm can apply **michalewicz1996genetic**. A **binary encoding** represents each gene with one bit, so every decision has two possible states. In compression, a bit value of 1 may indicate that a neuron, head, or channel is retained, while 0 indicates that it is removed. An **integer encoding** represents each gene with an integer selected from a finite set of choices. This is useful when a layer must choose one option among several possible bit-widths, such as $\{2, 4, 8, 16\}$.

The choice of encoding is important because it determines what the genetic operators can change. A poor encoding may create invalid candidates or make useful solutions hard to reach. A good encoding reflects the structure of the compression problem while keeping the search space manageable. Once this representation is fixed, the algorithm can operate on a population across generations.

Operation of Genetic Algorithms

A GA begins by initializing a population, either randomly or from existing candidate designs. Each individual is evaluated using the fitness function. The algorithm then selects promising candidates, recombines their encoded decisions through crossover, and applies mutation to preserve exploration **goldberg1989genetic**. The resulting candidates form the next population, and this cycle is repeated until a stopping criterion is reached, such as a maximum number of generations, convergence of the population, or an evaluation budget. The three operators below define how this update takes place.

Selection Operator. Selection chooses which individuals are allowed to reproduce. Better candidates should have a higher chance of being selected, but weaker candidates are not always discarded immediately because they may contain useful partial structures. Common strategies include roulette-wheel selection, rank-based selection, and tournament selection **baker1985adaptive**; **miller1995genetic**. The selected candidates then serve as parents for crossover.

Crossover Operator. Crossover combines information from two or more parent individuals to produce offspring. In binary or integer encodings, this may exchange segments of chromosomes between parents. In uniform crossover, each gene can be inherited independently from either parent **syswerda1989uniform**. For compression, crossover can combine useful pruning patterns or precision assignments discovered in different candidates. Since crossover reuses information already present in the population, mutation is used to introduce local changes.

Mutation Operator. Mutation introduces small random changes into an individual. It prevents the population from collapsing too early around one region of the search space and allows

new structures to appear **back1993optimal**. In a compression chromosome, mutation may flip a keep-or-remove bit, change the bit-width of a layer, or adjust a pruning threshold.

With these mechanics defined, the compression problem can now be written as a genetic search problem.

1.9.2 Compression as a Search Problem

With the GA terminology in place, compression can be described as a search over encoded model variants. Let $\mathbf{x}$ denote a compression configuration, such as a pruning mask, a vector of layer-wise bit-widths, or a set of structural choices. Evaluating $\mathbf{x}$ produces a compressed model whose quality depends on both predictive performance and deployment cost.

In pruning, this means that a candidate can encode which Transformer structures remain active. For example, a chromosome may represent retained attention heads, selected FFN widths, or removed layers. The search procedure then compares complete candidates rather than isolated pruning decisions, which is important when the quality of one removal depends on the rest of the architecture.

1.10 Conclusion

This chapter established the mathematical and architectural foundations required for the study of compression methods in Chapters ?? and ?. It introduced the neuron model, activation functions, multilayer perceptrons, learning objectives, gradient-based optimization, and back-propagation, then described the Transformer architecture as the primary model targeted by the pruning methods discussed in this report.

The discussion of hardware-aware efficiency formalized the principal deployment metrics that compression methods seek to improve, including parameter count, memory footprint, arithmetic cost, bandwidth demand, latency, and throughput. The chapter concluded by introducing genetic algorithms and multi-objective search as a framework for exploring discrete design spaces with competing objectives, which serves as the basis for the proposed contribution rather than for the pruning review that follows.

Within this conceptual frame, pruning emerges as a natural next step. Once neural computation is expressed through parameterized operators, decomposed into identifiable architectural units, and evaluated through resource-sensitive metrics, the central pruning question becomes precise: which parts of the network may be removed with limited functional loss, and under what deployment constraints does that removal become meaningful.

Chapter 2

Neural Network Pruning

2.1 Introduction

Within the framework established in Chapter ??, pruning can be understood as a constrained reduction of a trained network. The optimization perspective defines how pruning error is measured, the architectural structure of the network determines which components can be removed, and deployment metrics determine whether pruning leads only to nominal sparsity or to practical savings.

Pruning is an established deep neural network compression technique. Its purpose is to remove non-critical active parameters or operations in a network in a way that preserves the overall behavior of the pruned network to be similar to a reference network. In this removal process, the actual topology of the network is modified by setting weights, channels, filters, layers, or other units to zero, by masking them, or by removing them completely. The mask decision is simultaneously a compression choice and an architectural one.

The relevance of pruning follows from the over-parameterised nature of modern neural networks. Large models often contain substantial redundancy distributed unevenly across layers and parameter groups [denil2013predicting](#); [michel2022not](#). Table ?? illustrates the scale: each parameter requires 4 bytes at full precision (FP32) or 2 bytes at half precision (FP16), placing a hard memory floor on deployment before activations or caches are considered [He2016](#); [devlin2019bert](#); [radford2019language](#); [touvron2023llama2](#); [achiam2023gpt](#).

Table 2.1: Parameter counts and memory footprints for selected modern models. FP32 assumes 4 bytes per parameter; FP16 assumes 2 bytes per parameter.

Model	Params	FP32 (GB)	FP16 (GB)	Notes
ResNet-50	25.6 M	0.10	0.05	Image classification
BERT-Base	110 M	0.44	0.22	Encoder model
GPT-2 (XL)	1.56 B	6.24	3.12	Autoregressive model
LLaMA-2 (7B)	6.74 B	26.9	13.5	Large decoder model
LLaMA-2 (70B)	68.9 B	275	138	Requires multiple GPUs
GPT-4	–	–	–	Parameters not publicly disclosed

A simple parameter count shows why sparsity cannot be interpreted as savings by itself.

For example, a 70B-parameter model requires about 140 GB in FP16 under dense storage, and setting half of its weights to zero without any type of physical pruning does not change this footprint unless the representation or execution pattern also changes. Prior work on sparse neural network execution shows that practical acceleration depends on whether the sparse pattern is exploitable by the software and hardware stack [gale2020sparsegpu](#); [cheng2024survey](#). The pruning problem is therefore not only to reduce a numerical count of weights, but to choose which parameters or structures remain active under a given cost constraint. This leads us in the next section to formalize this choice using mask variables [kwon2022fast](#).

2.2 Theoretical Framework of Pruning

2.2.1 Formal Problem Formulation

Given a parameterized network $f(\cdot; \Theta)$ whose P parameters are stored in $\Theta \in \mathbb{R}^P$, a binary mask $M \in \{0, 1\}^P$ is chosen such that those parameters selected to be kept (whose mask value is 1) can be retrieved and those not (mask value 0) excluded from further calculation by computing $f(x; M \odot \Theta)$, which zeros out parameters for which $M_i = 0$ by elementwise product.

The goal is then to find the mask M and the remaining weights Θ that keep the loss $\mathcal{L}$ as low as possible while meeting a budget on how many parameters are retained:

$$\min_{\Theta, M} \mathcal{L}(M \odot \Theta) \quad \text{subject to} \quad \frac{\|M\|_0}{P} \leq \kappa, \quad (2.1)$$

where $\|M\|_0$ counts the nonzero entries of M and $\kappa \in (0, 1]$ is the target density. In practice this hard constraint is often replaced by a regularised objective that avoids having to fix κ in advance:

$$\min_{\Theta, M} \mathcal{L}(M \odot \Theta) + \lambda \Omega(M), \quad (2.2)$$

where λ controls how aggressively compression is penalised and $\Omega(M)$ can capture parameter count, memory footprint, or latency depending on the deployment target. This regularised form folds the budget into the objective instead of enforcing it as a hard constraint (Eq. (2.1)).

Finding an optimal pruning solution is generally NP-hard, since the search space grows exponentially with the number of parameters [dong2017learning](#). Practical methods thus resort to approximate measures which evaluate how pruning a parameter or structure affects performance without considering all possible masks. Such measures can be obtained by approximating the change in training loss due to pruning, commonly through a Taylor expansion [molchanov2017variational](#).

2.2.2 Taylor Expansion Analysis of Pruning Loss

The impact of pruning on the loss can be characterised by expanding the loss around the trained weights Θ^* . Let $\delta\Theta = M \odot \Theta^* - \Theta^*$ be the perturbation induced by masking:

$$\Delta\mathcal{L} \approx \underbrace{\nabla\mathcal{L}(\Theta^*)^\top \delta\Theta}_{\text{1st order}} + \underbrace{\frac{1}{2} \delta\Theta^\top H(\Theta^*) \delta\Theta}_{\text{2nd order}} + \mathcal{O}(\|\delta\Theta\|^3), \quad (2.3)$$

where $H(\Theta^*) \in \mathbb{R}^{P \times P}$ is the Hessian of the loss at Θ^* . The expansion separates the pruning effect into first-order sensitivity and second-order curvature (Eq. (2.3)). The first-order term measures how sensitive the loss is to the direction of weight removal: a weight with a large gradient, when removed, shifts the loss proportionally. What the second-order term adds is curvature. It captures how steeply the gradient itself changes as weights are zeroed out, which determines whether removing a small weight from a sharp loss valley is actually safe. At a stationary point of training where $\nabla\mathcal{L}(\Theta^*) \approx 0$, the first-order term vanishes and curvature becomes the decisive quantity.

Depending on which terms are retained and how H is approximated, this expansion gives rise to the full family of saliency criteria used in practice, from magnitude scoring (zeroth order, which ignores both terms and uses weight magnitude as a proxy) through gradient-based methods (first order) to blockwise curvature methods, each of which is developed in Section 2.2.2 and applied to specific methods in Chapter 2.3.

For second-order criteria, the same Taylor expansion can be used while keeping the curvature term. This gives a more accurate estimate of the loss change, but it also requires approximating the Hessian, which is expensive at modern scales. The original OBD criterion is a simple diagonal approximation, while later methods apply the same idea blockwise to recover tractability.

2.2.3 Optimal Brain Damage and Optimal Brain Surgeon

OBD **lecun1989optimal** is the simplest instantiation of the second-order term of the Taylor expansion: it assumes the Hessian is diagonal and applies no compensating correction to the remaining weights after the removal of a single weight, so the saliency of each weight reduces to

$$S_i^{\text{OBD}} = \frac{1}{2} H_{ii} \theta_i^2. \quad (2.4)$$

Under the OBD approximation, saliency becomes a Hessian-weighted square of the parameter (Eq. (2.4)). This makes ranking cheap, but it ignores interactions between weights. OBS **hassibi1993second** drops the diagonal assumption entirely: after each removal it solves for the optimal correction of the remaining weights induced by removing a weight, achieving strictly lower loss increase than OBD. The price is the full inverse Hessian, which is intractable at modern scales. This is where the connection to modern post-training compression becomes direct: later methods recover tractability by applying OBS-style updates blockwise, one layer at a time, with the resulting per-layer formula derived in Section 2.2.3 **frantar2023sparsegpt**; **frantar2022gptq**.

2.2.4 Density, Sparsity, and Effective Topology

The saliency criteria above produce a score for each weight, but before deciding which weights to remove it is useful to establish what a removal decision actually means quantitatively. Given N_{act} active (nonzero) parameters out of a total of P , the *density* and *sparsity* of the pruned network are

$$\rho = \frac{N_{\text{act}}}{P}, \quad s = 1 - \rho. \quad (2.5)$$

Density and sparsity define how many parameters remain active (Eq. (2.5)). They say nothing about where the removed weights were located. The same sparsity $s = 0.90$ can describe a network where zeros are scattered uniformly across all layers, leaving every operation dense but smaller, or one where zeros are concentrated in a few layers, leaving those layers nearly intact while others are almost entirely zeroed out, or one where the pattern is irregular enough that hardware cannot skip the zeroed operations without specialised sparse-format support **hoefer2021sparsity**. A sparsity ratio is therefore a necessary but insufficient description of a pruned network; what matters equally is the *structure* of the removal pattern. This is what the next section formalises.

2.3 Pruning Structures

The pattern in which weights are removed determines whether the resulting model can be accelerated on real hardware, and how much freedom the pruning criterion has in choosing which units to remove **cheng2024survey**; **hoefer2021sparsity**. Three structural regimes cover the main options, each sitting at a different point on the trade-off between pruning flexibility and deployment efficiency.

2.3.1 Unstructured Pruning

Unstructured pruning removes individual scalar weights without any constraint on their positions in the weight matrix **han2015deep**. Formally, given an importance score $S(\theta_i)$ for each parameter θ_i , the binary mask $M \in \{0, 1\}^P$ retains the top- κ fraction of weights by score:

$$M_i = \mathbf{1}[S(\theta_i) \geq \tau_\kappa], \quad (2.6)$$

where τ_κ is the $(1 - \kappa)$ quantile of the score distribution. This connects the density constraint to a concrete mask rule: keep the weights above the score threshold and zero out the rest (Eqs. (2.5) and (2.6)).

This regime offers the most freedom because each weight is scored independently, which tends to produce the best accuracy at a given sparsity level **frankle2018lottery**. The cost is deployment: the resulting zero pattern is irregular, and standard hardware cannot skip zeroed multiplications without sparse storage formats, such as compressed sparse row (CSR) formats that store only nonzero values and their indices, and specialised kernels that skip zero-valued operations. In practice, latency reductions are modest unless sparsity exceeds roughly 90%, and even then only when paired with sparse execution support **gale2020sparsegpu**.

2.3.2 Semi-Structured Pruning ($N:M$ Sparsity)

$N:M$ *sparsity* is a constrained form of unstructured pruning that imposes a regular local pattern **mishra2021accelerating**. Instead of allowing arbitrary weights to be removed anywhere in the matrix, the weight vector is partitioned into consecutive non-overlapping groups $\mathcal{G}$ of M elements. Within each group, exactly N elements are retained and the remaining $M - N$ are zeroed:

$$\|m_{\mathcal{G}}\|_0 = N \quad \forall \mathcal{G}, |\mathcal{G}| = M, \quad (2.7)$$

Here $m_{\mathcal{G}} \in \{0, 1\}^M$ is the mask restricted to group $\mathcal{G}$ and $\|\cdot\|_0$ counts its nonzero entries. The constraint forces each local group to keep the same number of weights, which makes the sparsity pattern more predictable than unconstrained unstructured pruning.

A widely used instance of semi-structured sparsity is the 2:4 pattern, where only two weights are kept in every group of four weights **pytorch2024semistructured**; **hu2024accelerating**. This gives the hardware a predictable sparse layout based on small weight groups. NVIDIA Ampere Sparse Tensor Cores support this fine-grained sparsity pattern and can exploit it for faster matrix multiplication when the required 2:4 constraint is satisfied **nvidia2020a100**; **mishra2021accelerating**.

After arbitrary weight removal and local group-wise constraints, a question that may come to mind is what happens if we try to remove coarser structures, such as complete architectural components. This is the topic of the next point.

2.3.3 Structured Pruning

Structured pruning removes entire vectors or submodules rather than individual weights, producing a strictly smaller dense model with no irregular sparsity **li2017pruning**. Because the result is a standard dense matrix, no sparse storage format or specialised kernel is needed; the compressed model runs on any hardware that supports ordinary matrix multiplication.

The removable unit depends on the architecture. In a convolutional network it is typically a filter or a channel; in a fully connected network it may be a neuron or a hidden dimension. The specific Transformer instantiation is covered later in Section ??.

Formally, removing rows and columns from a weight matrix $W \in \mathbb{R}^{m \times n}$ using a row mask $r \in \{0, 1\}^m$ and a column mask $c \in \{0, 1\}^n$ yields the submatrix

$$W' = W[\mathcal{R}, \mathcal{C}] \in \mathbb{R}^{m' \times n'}, \quad (2.8)$$

where $\mathcal{R} = \{i : r_i = 1\}$ and $\mathcal{C} = \{j : c_j = 1\}$ are the index sets of retained rows and columns, and $m' = |\mathcal{R}|$, $n' = |\mathcal{C}|$ are the reduced dimensions. The point of this construction is that structured pruning produces a smaller dense matrix, not a dense matrix with scattered zeros (Eq. (??)).

Figure ?? illustrates how the three regimes differ in their removal patterns. In the unstructured case, zeroed weights are scattered arbitrarily across the matrix, giving maximum freedom but producing no contiguous blocks that hardware can skip. Semi-structured sparsity resolves this by imposing a fixed retention ratio within each local group, aligning the pattern with ded-

icated hardware engines. Structured pruning goes further: removing entire rows or columns leaves a fully dense submatrix with no irregular zeros and no sparse-format overhead.

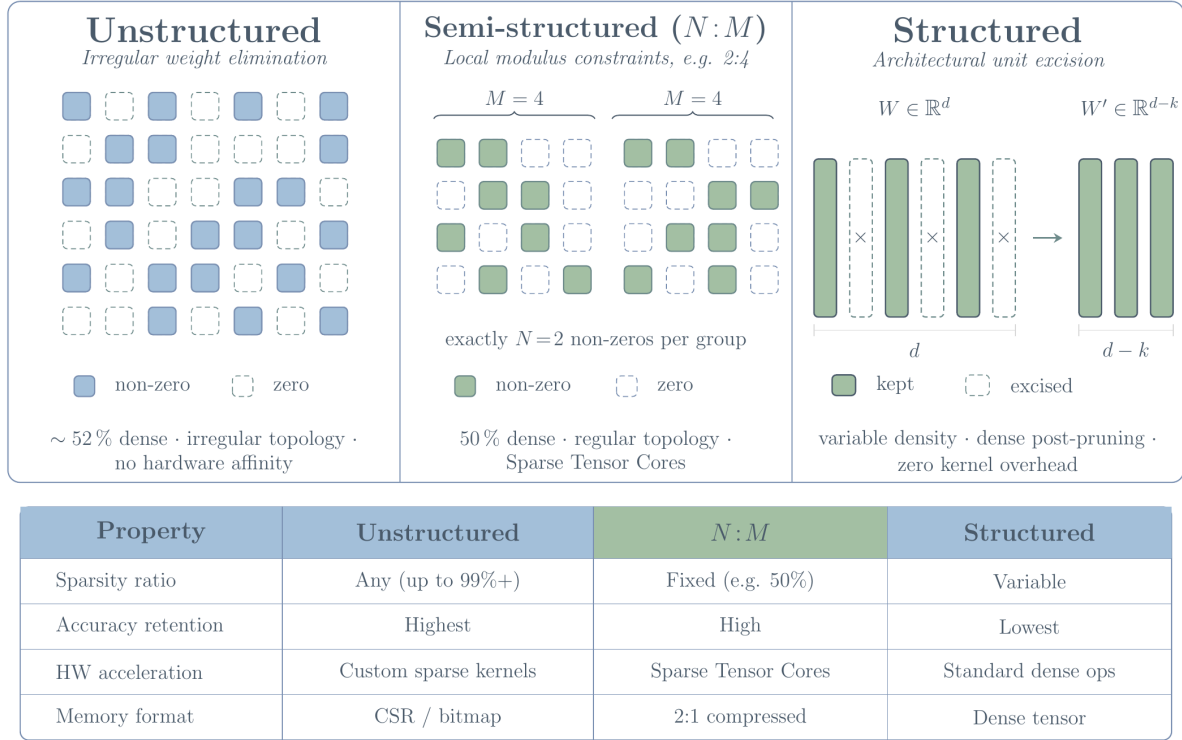


Figure 2.1: Comparison of unstructured, semi-structured, and structured sparsity patterns in a weight matrix. White cells indicate removed weights. Unstructured sparsity scatters zeros freely across the matrix; semi-structured sparsity enforces a fixed retention ratio within each local group; structured sparsity removes entire rows or columns, yielding a dense submatrix.

2.4 Pruning Criteria

While the sparsity structure determines the shape and hardware footprint of the compressed model, it says nothing about *which* specific weights or units should be removed. That decision is governed by the pruning criterion: given a fixed removal budget, the criterion ranks the parameters or structures by their estimated importance to the network’s output, so that the least important ones are removed first.

A **pruning criterion** assigns an importance score $S(\theta_i)$ (or $S(g)$ for a structured group g) that serves as a proxy for the loss change $\Delta\mathcal{L}$ induced by removing that element. The ideal criterion ranks units by *true irreplaceability*: the unit with the lowest true loss impact should be removed first.

2.4.1 Magnitude-Based Criteria

The most common zeroth-order criterion evaluates a weight by its absolute value **han2015learning**:

$$S_{\text{mag}}(w_i) = |w_i|. \quad (2.9)$$

For structured groups (e.g. a convolutional filter $W_g \in \mathbb{R}^{c_{\text{out}} \times c_{\text{in}} \times k \times k}$), the same idea is applied to the whole group rather than to one scalar weight. Common choices are the ℓ_1 , ℓ_2 , and ℓ_∞ norms:

$$S_g^{(1)} = \|W_g\|_1 = \sum_i |w_{g,i}|, \quad (2.10)$$

$$S_g^{(2)} = \|W_g\|_2 = \sqrt{\sum_i w_{g,i}^2}, \quad (2.11)$$

$$S_g^{(\infty)} = \|W_g\|_\infty = \max_i |w_{g,i}|. \quad (2.12)$$

These group scores extend the scalar magnitude rule to structured units: first score a whole tensor, then remove the groups with the smallest values. The ℓ_1 score measures total absolute magnitude, the ℓ_2 score measures Euclidean energy, and the ℓ_∞ score keeps only the largest entry in the group (Eqs. (??), (??), (??), and (??)).

The ℓ_2 norm is the most common because it is differentiable and rotation-invariant. The ℓ_∞ norm is sensitive to outliers and is less often used alone.

Interpretation. At a stationary point with a well-conditioned Hessian, $S_{2\text{nd}} = \frac{1}{2} H_{ii} w_i^2 \propto w_i^2$ if H_{ii} is approximately constant across parameters. Under this assumption, ranking by $|w_i|$ is equivalent to ranking by the second-order saliency **lecun1989optimal**. When Hessian diagonal entries vary greatly across layers (which they do in practice), magnitude alone may rank units from poorly-conditioned layers too aggressively.

2.4.2 Gradient-Based Criteria

First-order saliency estimates include the effect of the local gradient **molchanov2017variational**:

$$S_{\text{grad}}(w_i) = \left| \frac{\partial \mathcal{L}}{\partial w_i} \cdot w_i \right|. \quad (2.13)$$

This is obtained from the first-order Taylor term with $\delta w_i = -w_i$ (complete removal), which turns the local gradient into a gradient-weight saliency score (Eq. (??)). For structured pruning, the same criterion can be extended to a removable group by summing over all constituent parameters:

$$S_{\text{grad}}(g) = \left| \sum_{i \in g} \frac{\partial \mathcal{L}}{\partial w_i} \cdot w_i \right|. \quad (2.14)$$

This produces one importance value for the whole group, not one value per scalar weight (Eq. (??)) **molchanov2017variational**.

2.4.3 Second-Order Criteria

Diagonal approximation (OBD style).

$$S_{2nd}(w_i) = \frac{1}{2} H_{ii} w_i^2. \quad (2.15)$$

The diagonal curvature term used in this score can be estimated from the Gauss-Newton approximation to the Hessian, which requires computing per-sample gradients:

$$H \approx G^\top G, \quad G_{ij} = \frac{\partial \ell_j}{\partial w_i}, \quad (2.16)$$

where ℓ_j is the loss on sample j . The diagonal is then $H_{ii} \approx \sum_j G_{ij}^2$, giving the curvature factor used by the second-order score (Eq. (??)).

Blockwise inverse Hessian. The full OBS correction is too expensive to apply to all parameters at once, so modern methods apply the same idea locally inside large linear layers **hassibi1993second; frantar2023sparsegpt**. In a layer with weight matrix $W \in \mathbb{R}^{m \times n}$, the calibration activations are collected in $X \in \mathbb{R}^{n \times N}$, with one column per calibration sample. These activations give the empirical input Hessian $\hat{H} = XX^\top / N$, which describes how input directions interact inside the layer. The layer is then processed column by column: when one weight is removed, the remaining weights in the current block are adjusted so that the layer output changes as little as possible.

$$W'_{:,j} = W_{:,j} - \frac{w_{:,j}}{[\hat{H}^{-1}]_{jj}} w_{:,j}^\top \hat{H}_{j+1:,j}^{-1}, \quad (2.17)$$

This correction, written in Eq. (??), carries the pruning error into the surviving columns instead of leaving it concentrated at the removed weight. After each removal, the remaining inverse Hessian is updated efficiently, so the next pruning decision is made with the current block state rather than the original dense one. The total cost is $O(n^3 + mn^2)$ per layer.

2.4.4 Activation-Aware Criteria

Some criteria scale saliency by the activation statistics at the weight's input. Concretely, the weight magnitude is multiplied by the ℓ_2 norm of the input feature it processes **sun2023simple**:

$$S_{act}(W_{i,j}) = |W_{i,j}| \cdot \|x_j\|_2, \quad (2.18)$$

where $\|x_j\|_2$ is the ℓ_2 norm of the j -th input feature computed over a calibration set of N samples,

$$\|x_j\|_2 = \sqrt{\sum_{s=1}^N x_{s,j}^2}. \quad (2.19)$$

This equals $\sqrt{N}$ times the root-mean-square (RMS) of that feature's activations. RMS measures how much energy a feature carries on average across inputs. A feature with consistently large activations has high RMS. One that is rarely triggered has low RMS and contributes little to

the output regardless of its associated weight’s magnitude. Equation (??) makes the criterion sensitive to both: a small weight on a high-RMS feature scores higher than a larger weight on a near-silent one.

Activation magnitudes are distributed unevenly across input features **sun2023simple**; **wei2022outlier**. Magnitude-only scoring ignores this. It assigns identical importance to a weight on a high-energy feature and to an equally large weight on a near-zero feature, systematically underestimating the contribution of the first group. Scaling by $\|x_j\|_2$ corrects that bias directly.

2.4.5 Learnable and Adaptive Criteria

A fourth family introduces *trainable mask scores* $v \in \mathbb{R}^P$ that are optimised jointly with weights **louizos2018learning**; **mallya2018packnet**:

$$M_i = \sigma\left(\frac{v_i - \bar{v}}{\epsilon}\right), \quad \text{or} \quad M_i = \mathbf{1}[v_i \geq \tau], \quad (2.20)$$

The mask can be kept soft during training or converted into a hard keep/remove decision (Eq. (??)). The discrete threshold is approximated during backpropagation by a straight-through estimator **bengio2013estimating**, which passes the gradient through the threshold as if it were the identity. The learning objective then adds a sparsity penalty so that accurate but dense masks are discouraged:

$$\mathcal{L}_{\text{sparse}} = \mathcal{L}(\Theta, M(v)) + \lambda \|v\|_1. \quad (2.21)$$

This objective balances task performance against mask sparsity (Eq. (??)).

These approaches are valuable especially in the setting where the training distribution is accessible and the budget for mask optimisation is not the constraint. At moderate sparsities in fine-tuning, they outperform static criteria **sanh2020movement**; **louizos2018learning**.

2.4.6 Criterion Comparison

Table ?? places the criteria of this section side by side. Read from top to bottom, it traces a single trade-off: the cheaper criteria near the top observe only the weights themselves, while the richer ones below require gradients, curvature, activations, or a training signal. That extra information is precisely what buys the accuracy gains in the last column. The table is therefore best read as a map from *what a criterion is allowed to observe* to *how well it ranks units for removal*, which is the trade-off a practitioner balances against the available data and compute budget.

Table 2.2: Summary comparison of pruning criteria.

Criterion	Information used	Data needed	Grad?	H?	Typical accuracy
Magnitude	Weight values only	None	No	No	Baseline
Gradient (Taylor)	Gradient $\times$ weight	Train/calib	Yes	No	+1–3% over mag
OBD (§??)	Diag. curvature	Train/calib	Yes	Diag	+1–5% over mag
Blockwise OBS (§??)	Full curvature per layer	Calibration	No	Block	Best (post-train)
Activation-weighted	Magnitude $\times$ activation	Calibration	No	No	Strong post-train
Learned / trajectory mask	Score trajectory	Fine-tune	Yes	No	Best for fine-tune

2.5 Pruning Strategies

The criteria defined above state *what* to remove. A related question is *when* this removal occurs and when to let the model recover in between removals. A pruning schedule describes the pattern of mask updates and recoveries during the course of training or preparing for inference, which is what this section covers.

2.5.1 One-Shot Pruning

In *one-shot pruning*, importance scores are computed once from the fully trained model and all removal decisions are made in a single pass **han2015deep**. Algorithm ?? shows the procedure.

Algorithm 1 One-Shot Pruning

Input: Trained model $f(\cdot; \Theta)$, target density κ

Output: Pruned model $f(\cdot; M \odot \Theta)$

- 1 Compute $S(\theta_i)$ for all $i \in [P]$ Set threshold $\tau \leftarrow (1 - \kappa)$ quantile of $\{S(\theta_i)\}$ $M_i \leftarrow \mathbf{1}[S(\theta_i) \geq \tau]$
Optional: fine-tune Θ with M fixed **return** $M \odot \Theta$
-

One-shot pruning requires no additional training rounds and can be applied directly to a pretrained model, which makes it simple to run. But that simplicity has a cost. All removal decisions come from a single pass over the loss landscape on a small calibration set, with no opportunity for the model to recover between mask updates. Accuracy at a given sparsity is lower than for iterative methods **han2015deep**; **zhu2018prune**.

2.5.2 Iterative Pruning and Recovery

Iterative pruning directly addresses the limitation of one-shot pruning by interleaving mask updates with weight recovery steps **zhu2018prune**. Instead of removing all targeted weights at once, the model is pruned gradually over R rounds, with a fine-tuning phase after each round to let the remaining weights compensate for what was removed. Algorithm ?? shows a common formulation using a cubic sparsity schedule.

Algorithm 2 Iterative Pruning

Input: Trained model Θ , target density κ , rounds R

Output: Pruned model $M \odot \Theta$

```

2  $\kappa_0 \leftarrow 1.0$  // start fully dense
3 for  $r = 1$  to  $R$  do
4    $\kappa_r \leftarrow \text{SCHEDULE}(\kappa, r, R)$  // interpolate density toward target
5   Compute scores  $S(\theta_i)$  on current  $\Theta$  // re-evaluate importance
6   Update mask  $M$  to density  $\kappa_r$  // remove lowest-scoring units
7   Fine-tune  $\Theta$  with  $M$  fixed for  $T_r$  steps // recover accuracy
8 end
9 return  $M \odot \Theta$ 

```

The SCHEDULE function can take many forms such as linear, cubic, cosine, or step, and the choice is often borrowed directly from learning rate scheduling practice, since both problems share the same goal of controlling how aggressively a quantity changes over the course of training **zhu2018prune**. Figure ?? illustrates several common variants.

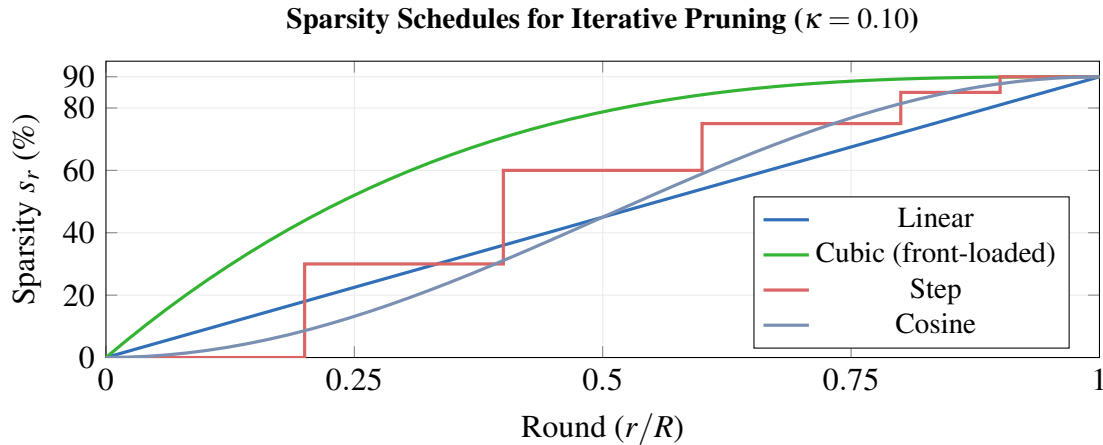


Figure 2.2: Sparsity schedules for iterative pruning targeting 90% sparsity over R rounds. Cubic and cosine schedules slow down near the target, giving the model time to stabilise before the final mask is frozen; linear and step schedules do not.

Linear growth applies equal increments each round, so the removal rate at the end of training is the same as at the start, with no settling period at high sparsity. Cubic growth concentrates most of the pruning early: by $r/R = 0.5$ roughly 87% of the target sparsity budget is already spent, and the remaining rounds operate at near-final density with small incremental changes. Step schedules skip continuous interpolation and instead apply discrete jumps at fixed intervals, holding the mask constant between updates. Cosine growth starts slowly and accelerates toward the midpoint, then decelerates as sparsity approaches the target. The last rounds under cosine resemble those of cubic. Both avoid committing large removals at high sparsity, where each pruned weight carries a larger marginal effect on the loss **zhu2018prune**.

2.5.3 Pruning During Training

A third family integrates sparsity directly into the optimisation loop by adding a regularisation term to the training loss, typically an ℓ_1 penalty of the form $\lambda \sum_i |\theta_i|$ **liu2017learning**. This encourages weights toward zero throughout training, so that sparsity emerges gradually rather than being imposed after the fact **han2015deep; guo2016dynamic**. Weights that fall below a threshold are later hard-pruned. The main requirement is access to the full training distribution, which makes this approach most suitable when training from scratch or during a supervised fine-tuning stage, rather than for post-hoc compression of a frozen model.

2.5.4 Post-Training Pruning

Post-training pruning applies pruning to a frozen pretrained model using only a small calibration set, without any gradient updates to the original training loss **frantar2023sparsegpt**. This makes it the practical default for large pretrained networks, where full retraining over the original training distribution is prohibitively expensive.

The procedure runs in four steps. First, a small calibration set of a few hundred samples is forwarded through the frozen model to collect layer-wise input activation statistics. Second, those statistics are combined with the weight values to compute a saliency score $S(\theta_i)$ for each weight or group. Third, the lowest-scoring units are zeroed out to produce the binary mask M^* . Fourth, an optional weight correction step adjusts the values of the remaining weights in each layer to compensate for the damage caused by the removals, an idea inherited from OBS and applied blockwise in modern post-training methods **frantar2023sparsegpt**. Figure ?? summarises these steps.

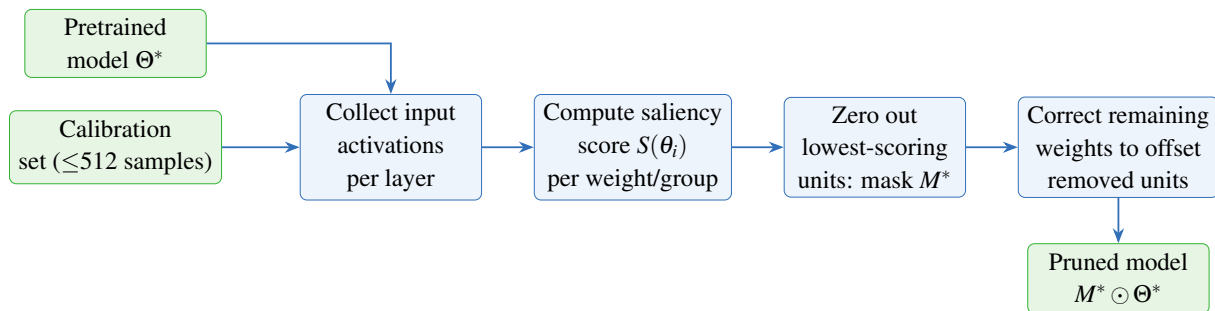


Figure 2.3: Post-training pruning pipeline. A small calibration set is forwarded through the frozen model to collect layer-wise activation statistics. These are used to score each weight or group, after which the lowest-scoring units are zeroed out. An optional weight correction step adjusts the remaining weights to compensate for the removed ones. No gradient update to the pretraining loss is performed at any stage.

So far, we have described pruning by means of three components: what structure should be eliminated, according to which evaluation measure, and which application strategy should

be adopted. Because our specific interest is Transformers, these components now have to be identified within the architecture being compressed. In the next section, we therefore describe an instantiation of the pruning approach within Transformer models.

2.6 Pruning Transformers

The general framework described so far identifies three decisions: which units to remove, by what criterion, and according to which schedule. Applying this to a Transformer means mapping each decision to the specific computational structures of the architecture. The three main removable units are attention heads, FFN channels, and entire Transformer layers. Each carries a different cost and requires a different adaptation of the pruning formulas.

2.6.1 Attention Head Pruning

The multi-head attention module computes attention in H parallel subspaces called heads. Given an input sequence $X \in \mathbb{R}^{n \times d}$, where n is the sequence length and d is the model dimension, each head h projects X into query, key, and value spaces of dimension $d_h = d/H$ using learned matrices $W_{Q_h}, W_{K_h}, W_{V_h} \in \mathbb{R}^{d \times d_h}$, computes scaled dot-product attention, and feeds the result through the shared output projection $W_O \in \mathbb{R}^{d \times d}$.

A head mask $m \in \{0, 1\}^H$ selects which of the H heads contribute to the output:

$$\text{MHA}_m(X) = \text{Concat}(m_1 \text{head}_1(X), \dots, m_H \text{head}_H(X)) W_O. \quad (2.22)$$

Setting $m_h = 0$ zeros out W_{Q_h} , W_{K_h} , W_{V_h} , and the corresponding block of columns in W_O (Eq. (??)). Since each head uses query, key, value, and output projections, the number of parameters removed per head is:

$$\Delta P_{\text{head}} = 4d d_h = \frac{4d^2}{H}. \quad (2.23)$$

Early work on head importance found that a large share of heads can be removed with no statistically significant drop in performance **micheel2022not**; **voita2019analyzing**. Heads that carry out identifiable functions, such as attending to adjacent tokens or tracking syntactic dependencies, are harder to remove, while heads whose attention patterns closely resemble those of other heads in the same layer are the most dispensable **voita2019analyzing**. Importance varies considerably across layers and no single head is universally critical **micheel2022not**.

2.6.2 FFN Channel Pruning

The feed-forward network in each Transformer layer expands the model dimension by a factor r before projecting back. For an input $X \in \mathbb{R}^{n \times d}$ and expansion ratio r , the standard FFN applies:

$$\text{FFN}(X) = \phi(XW_1 + b_1) W_2 + b_2, \quad (2.24)$$

where $W_1 \in \mathbb{R}^{d \times rd}$, $W_2 \in \mathbb{R}^{rd \times d}$, and ϕ is a pointwise activation function. This is the standard two-projection FFN block used before channel pruning (Eq. (??)). A channel mask $g \in \{0, 1\}^{rd}$

selects which of the rd intermediate neurons to retain:

$$\text{FFN}_g(X) = \phi(XW_1 + b_1) \text{diag}(g) W_2 + b_2. \quad (2.25)$$

Writing $\mathcal{K} = \{k : g_k = 1\}$ for the retained channel indices, the masked FFN can be rewritten as a dense FFN of smaller width:

$$\text{FFN}_g(X) = \phi(XW_{1,\mathcal{K}} + b_{1,\mathcal{K}}) W_{2,\mathcal{K},:} + b_2. \quad (2.26)$$

Removing one channel eliminates one row of W_1 and one column of W_2 , which is exactly what the masked and compact forms express (Eqs. (??) and (??)). Each removed channel therefore saves $2d$ parameters. For $rd - |\mathcal{K}|$ removed channels the total saving is $(2d)(rd - |\mathcal{K}|)$.

2.6.3 Layer Pruning

Each Transformer block is an additive function: $F_l(x) = x + F_l^{(1)}(x)$, where $F_l^{(1)}(x)$ is the composition of the attention sub-layer and the FFN sub-layer. When $F_l^{(1)}(x)$ is small relative to x , the residual connection carries the bulk of the signal and the block contributes little to the representation. Removing it leaves downstream activations largely unchanged.

A natural way to measure this is to compare the magnitude of $F_l(x)$ to the magnitude of the input x over a calibration set. Using the Frobenius norm $\|\cdot\|_F$ (the square root of the sum of all squared entries of a matrix) as the measure of size, the relative block contribution is:

$$S_l^{\text{block}} = \frac{\|F_l(X)\|_F}{\|X\|_F}, \quad (2.27)$$

where the norms are computed over a batch of calibration inputs X . A block with a small relative contribution is close to an identity map, so it becomes a natural candidate for removal (Eq. (??)) **micheel2022not; kurtic2023ziplm**.

Layer pruning has the coarsest effect of any structured pruning decision since a single removal eliminates all parameters in one attention module and one FFN, a far larger parameter count than removing a head or a set of channels. This makes it effective for reducing inference latency, which scales with the number of layers, but recovery through fine-tuning is generally needed when more than a small number of layers are removed **kurtic2023ziplm**.

2.7 Conclusion

The chapter formalized pruning as a constrained optimization problem, surveyed the three structural sparsity regimes, and compared the saliency criteria from magnitude scoring through second-order blockwise methods. Section ?? covers the Transformer instantiation.

Pruning strategy determines when the mask is updated and how much recovery the model gets between removals. One-shot, iterative, training-time, and post-training regimes each carry a different accuracy-cost profile, and the choice matters as much as the criterion. Together these components provide the vocabulary the method-level review in Chapter ?? builds on.

Sparsity topology, pruning unit, saliency signal, recovery procedure, and deployment objective each mark a distinct axis of variation. Methods that look similar on parameter count can differ on all five. The review in Chapter ?? is organized along these dimensions.

Chapter 3

Related Work

3.1 Introduction

Chapter ?? established the formal pruning problem, the main sparsity regimes, the dominant saliency criteria, and the Transformer structures that can be removed in practice. Against this background, the pruning literature may be read as a sequence of increasingly structured responses to the same core problem: how to remove computation while preserving the behavior of the reference model under explicit deployment constraints.

The literature is not homogeneous: some methods target weight-level sparsity, others remove executable structures such as heads, FFN channels, or layers, and others treat pruning as a constrained configuration-search problem under hardware or latency objectives. The discussion is therefore organized by pruning mechanism and interpreted with attention to the evaluation regime used by each paper, since compression level alone is insufficient to compare methods that optimize different units, use different recovery pipelines, or report different forms of efficiency.

3.2 Related Work on Pruning

Pruning is the most widely studied form of model compression, and it has been applied to Transformers in a variety of ways. The related work discussion is organized along two coupled axes: the topology of the induced sparsity pattern and the optimization signal used to identify removable units. This framing keeps the deployment question explicit from the outset, because similar nominal compression ratios may correspond to different execution regimes, different preprocessing costs, and different forms of realized speedup.

3.2.1 Problem Setting and Taxonomy

Building on the formal framework introduced in Chapter ??, the related-work discussion is organized along two coupled axes: the topology of the induced sparsity pattern and the optimization signal used to identify removable units.

Taxonomy of Transformer Pruning Methods

The main sparsity topologies were introduced formally in Section ?? (see Figure ??). From a deployment standpoint, they differ in whether pruning yields compressed storage alone or native dense-shape acceleration after model surgery.

The taxonomy adopted here follows four coupled axes. The *removable unit* determines whether the method acts on scalar weights or executable structures such as attention heads, FFN channels, and layers **micheel2022not**; **kurtic2022optimal**. The *optimization signal* ranges from magnitude and activation statistics to differentiable mask learning, first-order Taylor criteria, second-order curvature surrogates, or search-based objectives **sun2023simple**; **xia2022structured**; **ma2023llmpruner**; **frantar2023sparsegpt**; **kwon2022fast**. The *recovery stage* distinguishes direct post-training pruning from methods requiring LoRA recovery, gradual fine-tuning, or distillation. The *deployment objective* determines whether the reported benefit is compressed storage, dense-shape acceleration, or measured speedup **kurtic2023ziplm**; **evopress2024**; **darwinlm2025**.

Each family corresponds to a different deployment objective. Unstructured methods such as Wanda and SparseGPT primarily optimize weight-level saliency **sun2023simple**; **frantar2023sparsegpt**. Structured methods span differentiable mask-learning (CoFi), dependency-aware gradient methods (LLM-Pruner), Fisher-based post-training frameworks, and runtime-aware second-order methods (ZipLM) **xia2022structured**; **ma2023llmpruner**; **kwon2022fast**; **kurtic2023ziplm**. Search-based methods such as EvoPress and DarwinLM treat pruning as configuration optimization under explicit budget or hardware constraints **evopress2024**; **darwinlm2025**. Figure ?? and Table ?? summarize this organization.

Chapter 3. Related Work

Table 3.1: Taxonomy used to organize the pruning methods discussed in this chapter. The table distinguishes the removable unit, sparsity topology, optimization signal, recovery stage, and primary deployment claim for each method family.

Method	Unit	Topology	Signal	Recovery	Primary deployment claim
Wanda	Weight	Unstructured	Activation-aware saliency	None	Storage-oriented sparsity with strong zero-shot perplexity retention
SparseGPT	Weight	Unstructured	Single-weight second-order compensation	None	High-sparsity pruning with local reconstruction
CoFi	Layers / heads / FFN / hidden dimensions	Structured	Learned hard-concrete masks with L_0 control and distillation	Full task-specific pruning and fine-tuning	Dense-shape acceleration through jointly pruned Transformer structure
LLM-Pruner	Heads / channels / blocks	Structured	First-order Taylor scoring with dependency groups	LoRA recovery	Structural compression with dense-shape execution
Fast post-training pruning	Heads / FFN filters	Structured	Fisher-based constrained mask optimization	None beyond mask tuning	FLOPs- or latency-constrained structured pruning without retraining
ZipLM	Heads / FFN dimensions	Structured	Second-order structured saliency with runtime-constrained search	Gradual fine-tuning and distillation	Measured speedup under device-specific constraints
EvoPress / DarwinLM	Layer-wise compression levels	Heterogeneous structured	Proxy-guided or evolutionary search	Search-time evaluation or proxy training	Global configuration search under budget and hardware constraints

Figure ?? should be read as a map of the chapter rather than as a ranking of methods. It separates the pruning literature by the kind of object being removed and by the signal used to decide removal. Table ?? then makes this map concrete by assigning each reviewed method to a pruning unit, topology, scoring mechanism, recovery regime, and deployment claim.

The following subsections use this organization to review the selected pruning families in sequence: saliency-based methods, gradient-based structured methods, second-order methods, and search-based methods.

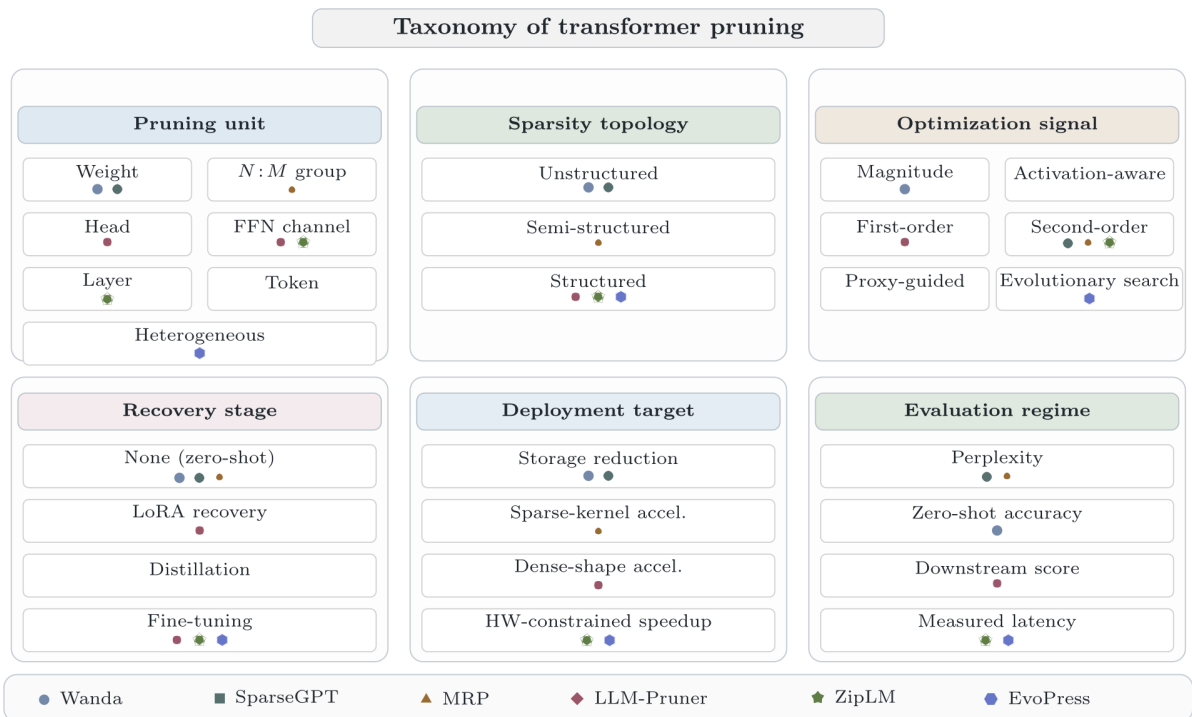


Figure 3.1: Taxonomy of the pruning methods considered in this chapter. The classification crosses the pruning unit, sparsity topology, optimization signal, recovery stage, deployment objective, and evaluation regime.

3.2.2 Saliency-Based Methods

Recall from Section ?? that the Taylor expansion of the loss change induced by pruning a weight θ_i produces a first-order term proportional to the gradient and a second-order term capturing curvature (Equation (??)). Saliency-based methods discard both terms and instead use the weight magnitude or a proxy derived from activation statistics as a surrogate for the true loss change. This zeroth-order approximation requires no gradient computation and no calibration beyond a simple forward pass, making these methods the cheapest option in the taxonomy.

Magnitude and Activation-Aware Baselines

Weight-Magnitude Pruning. Magnitude pruning uses absolute weight values as the saliency proxy:

$$\mathcal{S}_{i,j}^{\text{mag}} = |W_{i,j}|. \quad (3.1)$$

Its core assumption is that small weights contribute less to the output and can be removed with limited degradation. The method requires no gradient-based recovery and remains the simplest baseline against which more sophisticated algorithms are evaluated.

Wanda (Weights $\times$ Activations). Wanda `sun2023simple` refines this baseline by incorporating activation information from a small calibration set. The saliency score is

$$\mathcal{S}_{i,j}^{\text{wanda}} = |W_{i,j}| \cdot \|x_j\|_2, \quad (3.2)$$

where x_j is the j -th input activation channel and $\|x_j\|_2$ is its RMS norm estimated over N calibration samples. The key insight is that importance is jointly encoded in weight magnitude and the energy of the coupled input channel.

Algorithm Overview

Both magnitude pruning and Wanda are one-shot post-training methods in the regime of Section ??: scores are computed once and no recovery training is applied afterward. Wanda processes each layer independently: it collects input activations on a small calibration set (typically 128 samples), computes per-weight saliency scores as the product of weight magnitude and activation norm, then prunes the lowest-scoring weights for each output neuron until the target sparsity p is reached. No weight update or recovery step is applied. Magnitude pruning is a pure linear scan $O(|W|)$ with no calibration; Wanda adds only an activation-collection pass, keeping the overall cost near-linear.

These saliency methods are useful because they are cheap and easy to apply, but their decisions remain local. Once pruning moves from individual weights to heads, FFN channels, or layers, the method must account for structural dependencies between parameters. This motivates gradient-based structured pruning methods, where the score is computed over groups that can be removed consistently.

3.2.3 Gradient-Based Methods

Retaining the first-order term of the Taylor expansion (Equation (??)) gives a score proportional to the product of the gradient and the weight value. This makes the criterion sensitive to the direction of the loss landscape rather than just the weight magnitude, and it naturally extends to groups of weights by summing or aggregating the per-element scores. The methods in this section use exactly this first-order signal, applied to structured units such as heads, channels, and blocks rather than individual scalars.

Structured pruning replaces individual-weight saliency with group-level importance, because the natural units in a Transformer, namely attention heads, FFN channels, and entire layers, are coupled through residual pathways and shared projection matrices. Removing one head, for example, requires deleting consistent slices from W_Q , W_K , W_V , and W_O together; removing an FFN channel removes coordinated rows and columns from the two projection matrices. Any pruning criterion applied at this level must therefore identify groups that can be removed as a unit without breaking the forward pass **micheel2022not**; **kurtic2022optimal**.

LLM-Pruner. LLM-Pruner **ma2023llmpruner** addresses the dependency problem explicitly by constructing a graph over the neurons of the network. For neurons N_i and N_j with input set $\text{In}(N)$ and output set $\text{Out}(N)$, the graph encodes two rules:

$$N_j \in \text{Out}(N_i) \wedge \deg^-(N_j) = 1 \implies N_j \text{ depends on } N_i, \quad (3.3)$$

$$N_i \in \text{In}(N_j) \wedge \deg^+(N_i) = 1 \implies N_i \text{ depends on } N_j. \quad (3.4)$$

Here $\deg^-(N_j)$ is the in-degree of N_j (number of neurons feeding into it) and $\deg^+(N_i)$ is the out-degree of N_i . The transitive closure of these rules groups all coupled parameters into a single dependency group $\mathcal{G} = \{W_i\}_{i=1}^M$, which is the minimal unit that can be consistently removed.

Once groups are defined, each is scored using a first-order Taylor approximation of the loss increase over a small calibration set $\mathcal{D}$. The importance of a structure tensor W_i is estimated as

$$I_{W_i} \approx \left| \frac{\partial \mathcal{L}^\top(\mathcal{D})}{\partial W_i} W_i - \frac{1}{2} W_i^\top H W_i \right|, \quad (3.5)$$

where H is the local Hessian. The first-order gradient is retained alongside the Hessian term because the calibration set is only a proxy for the pretraining distribution; discarding it would introduce bias. An element-wise variant replaces H with a Fisher-diagonal approximation. Individual tensor scores within a group are aggregated by sum, product, max, or last-only operators, and groups with the lowest aggregate score are removed first.

LLM-Pruner sits between a post-training and a recovery regime: scoring is post-training and requires no gradient propagation through the full model, but the pruned network is then passed through a LoRA recovery stage in line with the iterative recovery approach of Section ?? . LoRA adapters **hu2021lora** are inserted into all linear modules and fine-tuned for two epochs over roughly 50K samples. The low-rank factors are then merged back into the weight matrices, leaving no adapter overhead at inference time. The full scoring stage requires only 10 short calibration sequences, making the preprocessing cost modest relative to the recovery phase.

CoFi. CoFi **xia2022structured** is the main structured-pruning example of the learnable-mask criteria introduced in Section ?? . It places binary masks on attention blocks, heads, FFN blocks, intermediate channels, and the shared hidden dimension, then optimises these masks jointly with the task loss:

$$\text{MHA}(X) = z_{\text{MHA}} \sum_{i=1}^{N_h} z_{\text{head}}^{(i)} \text{Att}\left(W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)}, X\right), \quad (3.6)$$

$$\text{FFN}(X) = z_{\text{FFN}} \text{gelu}(XW_U) \text{diag}(z_{\text{int}}) W_D, \quad (3.7)$$

where N_h is the number of attention heads, $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}, W_O^{(i)}$ are the query, key, value, and output projection matrices of head i , and W_U, W_D are the up- and down-projection matrices of the FFN. Each mask variable z is relaxed to a continuous value during training using the hard-concrete distribution for L_0 regularisation, which allows gradients to flow through the mask. The training objective combines task loss with a distillation term and a Lagrangian sparsity constraint:

$$\mathcal{L}_{\text{prune}} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{distil}} + \lambda_1 (\hat{s} - s_0) + \lambda_2 (\hat{s} - s_0)^2, \quad (3.8)$$

where s_0 is the sparsity target and $\hat{s}$ is the expected sparsity under the current mask distribution. Because the masks at different granularities interact, CoFi can discover structured subnetworks that a simple per-group ranking would miss.

Algorithm Overview

CoFi is a training-time iterative method in the sense of Section ?? : mask decisions and weight updates are interleaved throughout training rather than applied once. A warm-up phase first stabilizes the model weights before mask learning begins, with the sparsity constraint applied softly. The main phase then jointly optimises masks and weights under the Lagrangian objective, progressively tightening the sparsity target toward s_0 over training. Once the target is reached, each continuous mask variable is thresholded against zero to produce a binary decision, and a final fine-tuning pass is run on the surviving subnetwork. Because masks at all granularities are trained simultaneously, the discovered subnetwork reflects cross-level interactions rather than independent per-unit decisions.

Gradient-based structured pruning improves the consistency of the removed units, but the score still depends on a local approximation of how removal affects the loss. When pruning becomes more aggressive, interactions among remaining and removed weights become more important. This leads to second-order methods, which use curvature information or local reconstruction objectives to compensate for the error introduced by pruning.

3.2.4 Second-Order Methods

The second-order term of the Taylor expansion (Equation (??)) captures the curvature of the loss around the current weights. Retaining this term, rather than discarding it as saliency and

gradient methods do, allows the pruning criterion to account for how much a removal actually shifts the loss given the local geometry of the loss landscape. As discussed in Section ??, this leads to the OBS framework, where the optimal weight correction after a removal is derived from the inverse Hessian. The methods in this section apply that same principle at the scale of modern LLMs, recovering tractability by restricting the Hessian computation to individual layers or column blocks.

Optimal Brain Compression Formulation

For a layer with weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and calibration activations $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$, the layer-wise sparse reconstruction problem is

$$\min_{\mathbf{W}^{\text{sparse}}} \|\mathbf{W}\mathbf{X} - \mathbf{W}^{\text{sparse}}\mathbf{X}\|_F^2 \quad \text{s.t.} \quad \|\mathbf{W}^{\text{sparse}}\|_0 \leq (1 - p)d_{\text{out}}d_{\text{in}}, \quad (3.9)$$

where $\|\cdot\|_F$ is the Frobenius norm, $\|\cdot\|_0$ counts the number of nonzero entries, and $p \in (0, 1)$ is the target sparsity ratio. Using the empirical Hessian proxy $\mathbf{H} = \mathbf{X}\mathbf{X}^T/n$, this becomes a curvature-aware reconstruction problem whose solution depends on $\mathbf{H}^{-1}$ to redistribute pruning error across remaining weights. The methods that follow differ in how they instantiate this objective: SparseGPT removes one scalar weight at a time, MRP removes an entire N:M group jointly, Fisher-based pruning applies curvature scoring to structured masks, and ZipLM extends the same principle to executable Transformer structures under runtime constraints.

Single-Weight Iterative Removal (SparseGPT)

SparseGPT **frantar2023sparsegpt** greedily prunes one weight at a time while applying compensation updates to remaining weights in the current block. This can be read as a blockwise version of the OBS correction introduced in Section ?? and formalised for large linear layers in Section ?. Its local objective at iteration t is

$$\min_{\delta w} \frac{1}{2} \delta w^T H \delta w \quad \text{s.t.} \quad (w + \delta w) \cdot e_t = 0, \quad (3.10)$$

where e_t is the one-hot selector for the pruned weight.

Algorithm Overview

SparseGPT is a post-training one-shot method in the regime of Section ??: pruning and compensation are applied in a single forward pass over the layers without any subsequent gradient-based recovery. It processes the weight matrix in column blocks of size B . For each block, it first constructs the regularized inverse Hessian $(\mathbf{X}\mathbf{X}^T + \lambda I)^{-1}$. Within each block, it iteratively identifies the weight with minimum local second-order loss, sets it to zero, then updates the remaining weights in that block using the corresponding column of $\mathbf{H}^{-1}$. After processing a block, lazy batch updates are applied to adjacent blocks. The method scales as $O(d_{\text{in}}^2 d_{\text{out}} + d_{\text{in}}^3/B)$ per layer, with a calibration budget of approximately 128 samples.

It is more instructive to understand SparseGPT as a weight-level, Hessian-compensated method for pruning: each decision to prune weights is made locally, and weights are updated to

make the layer output’s deviation as minimal as possible **frantar2023sparsegpt**. It is through this same lens that SparseGPT’s modification for semi-structured N:M sparsity can be understood, where the compensation method is kept and mask selection is constrained to local groups. If the block size is set to $B_s = M$, and from each group of M weights the N weights with the smallest OBS reconstruction score are selected for removal, the compensation becomes more localized to a block rather than the whole layer. Within that block, however, weights are still selected and compensated one at a time, so the compensation at each step does not account for the interactions among the other weights being simultaneously removed from the same group, which is the gap MRP directly addresses.

Joint Optimization for Multi-Weight Removal (MRP)

The MRP framework **zhao2024pruning** generalizes second-order pruning to N:M groups. For a vectorized weight block w and binary pruning mask M , where $M_i = 1$ indicates that weight i is pruned, the objective minimizes the second-order loss increase for the full group:

$$\min_{\delta w} \frac{1}{2} \delta w^T H \delta w + g^T \delta w \quad \text{s.t.} \quad (w + \delta w) \odot M = 0, \quad (3.11)$$

where g is the gradient of the loss with respect to w , and $\odot$ denotes elementwise multiplication. Solving the group jointly yields a compensation update for the retained weights:

$$\delta w^* = -H_{\bar{M}\bar{M}}^{-1} (g_{\bar{M}} + H_{\bar{M}M} w_M), \quad (3.12)$$

where M denotes pruned indices, $\bar{M}$ retained indices, and w_M the weight values at the pruned coordinates. This expression makes the role of the block Hessian explicit: in N:M pruning, the removed weights are not independent decisions, so compensation must be computed for the group rather than for a single scalar.

Algorithm Overview

MRP is a post-training one-shot method in the regime of Section ??: no gradient updates or recovery training are applied after pruning. It operates layer by layer. For each layer, the block Hessian is estimated from calibration activations. The weight tensor is partitioned into non-overlapping groups of M consecutive weights. For each group, the joint second-order problem is solved: the mask selecting the $M - N$ weights with the highest reconstruction cost is identified, and the compensation update δw^* is applied to the retained N weights before the next group is processed. Because the group is treated as a unit rather than a sequence of scalar removals, the method correctly accounts for interactions among simultaneously pruned coordinates within the block.

Figure ?? illustrates this difference. SparseGPT compensates after removing one scalar weight, whereas MRP compensates after removing a constrained local group. The figure is therefore used to distinguish scalar second-order correction from group-level second-order correction.

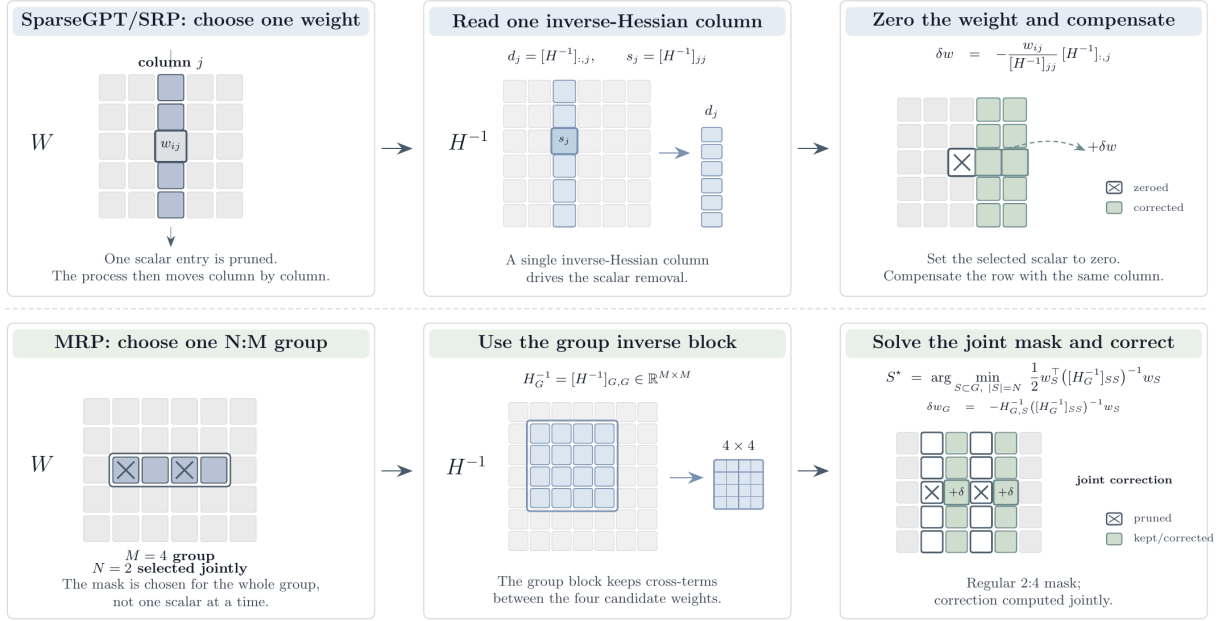


Figure 3.2: Second-order compensation strategies for unstructured and semi-structured pruning. SparseGPT removes one scalar weight per step and compensates using one column of $\hat{H}^{-1}$, while MRP removes an N:M group jointly and uses the block $(\hat{H}_{MM}^{-1})^{-1}$ to model interactions among simultaneously removed weights.

Fisher-Based Post-Training Structured Pruning

The post-training framework of Kwon et al. **kwon2022fast** applies second-order structured pruning without end-to-end post-pruning weight optimization. Binary masks over attention heads and FFN filters are found by solving

$$\arg \min_{\mathbf{m}} \mathcal{L}(\mathbf{m}) \quad \text{s.t.} \quad \text{Cost}(\mathbf{m}) \leq C, \quad (3.13)$$

where Cost is either a FLOPs or measured latency budget. Expanding around the dense mask and applying a diagonal Fisher approximation reduces the search to selecting units with the smallest diagonal Fisher scores subject to the cost constraint. For latency-constrained pruning, a piecewise-linear lookup table replaces the analytical cost model to incorporate real device behavior:

$$\text{LAT}(\mathbf{m}_\ell) = \begin{cases} 0, & \|\mathbf{m}_\ell\|_0 = 0, \\ c, & 0 < \|\mathbf{m}_\ell\|_0 \leq T, \\ a(\|\mathbf{m}_\ell\|_0 - T) + c, & \|\mathbf{m}_\ell\|_0 > T, \end{cases} \quad (3.14)$$

where c is the base latency incurred by any non-empty layer configuration, T is the structural threshold below which latency is approximately constant, and a is the slope capturing how measured latency grows beyond that threshold. All three constants are read from device benchmarks rather than derived analytically.

Algorithm Overview

The method proceeds in four steps: (1) diagonal Fisher scores are computed for head and FFN-filter masks on a small calibration set (2K training examples); (2) a FLOPs- or latency-constrained mask search is solved under the diagonal Fisher objective; (3) a block-diagonal Fisher refinement is applied layer by layer to recover interactions ignored by the diagonal approximation; (4) surviving mask values are tuned by layer-wise linear least-squares reconstruction. Because only mask values are updated, the method avoids end-to-end weight retraining. The paper reports pruning times of 39 seconds on GLUE and 135 seconds on SQuAD, compared to 33 hours for CoFi on MNLI.

Inference-Aware Structured Second-Order Pruning (ZipLM)

ZipLM **kurtic2023ziplm** applies an OBS-style reconstruction objective to executable Transformer structures rather than scalar weights. For a candidate structure S with column mask $\mathbf{M}_S$, the removal score is

$$\arg \min_S \sum_{i=1}^{d_{\text{row}}} \mathbf{W}_{i,\mathbf{M}_S} \left((\mathbf{H}^{-1})_{\mathbf{M}_S, \mathbf{M}_S} \right)^{-1} \mathbf{W}_{i,\mathbf{M}_S}^T, \quad (3.15)$$

where d_{row} is the output dimension of the weight matrix and the sum aggregates the reconstruction cost across all output rows. After removing a structure, the exact compensation and Hessian update are applied through block Gaussian elimination, so that all subsequent saliency scores remain consistent with the already-pruned state.

Inference-Aware Search

ZipLM adds a device-level objective on top of the local second-order scoring. It benchmarks attention and FFN configurations on the target hardware and stores measured runtimes in a lookup table. The global search then minimizes layer-wise structural sensitivity subject to a real speed constraint $\sum_{\ell} \tau_{\ell, s_{\ell}} \leq T_{\text{target}}$.

Algorithm Overview

ZipLM follows an iterative pruning-and-recovery regime in the sense of Section ??: pruning steps are interleaved with fine-tuning rounds rather than applied all at once. It operates in two phases (see Figure ??). In the global phase, attention-head and FFN configurations are benchmarked on the target environment and a layer-wise sparsity profile satisfying the speedup requirement is selected. In the local phase, each selected layer is compressed by iteratively scoring candidate structures with the structured saliency metric, applying the optimal weight compensation, and updating the inverse Hessian by block Gaussian elimination. Recovery follows a gradual prune-and-fine-tune schedule with combined task, logit, and token-level distillation losses, at a cost of 8–10 fine-tuning epochs per pruning step.

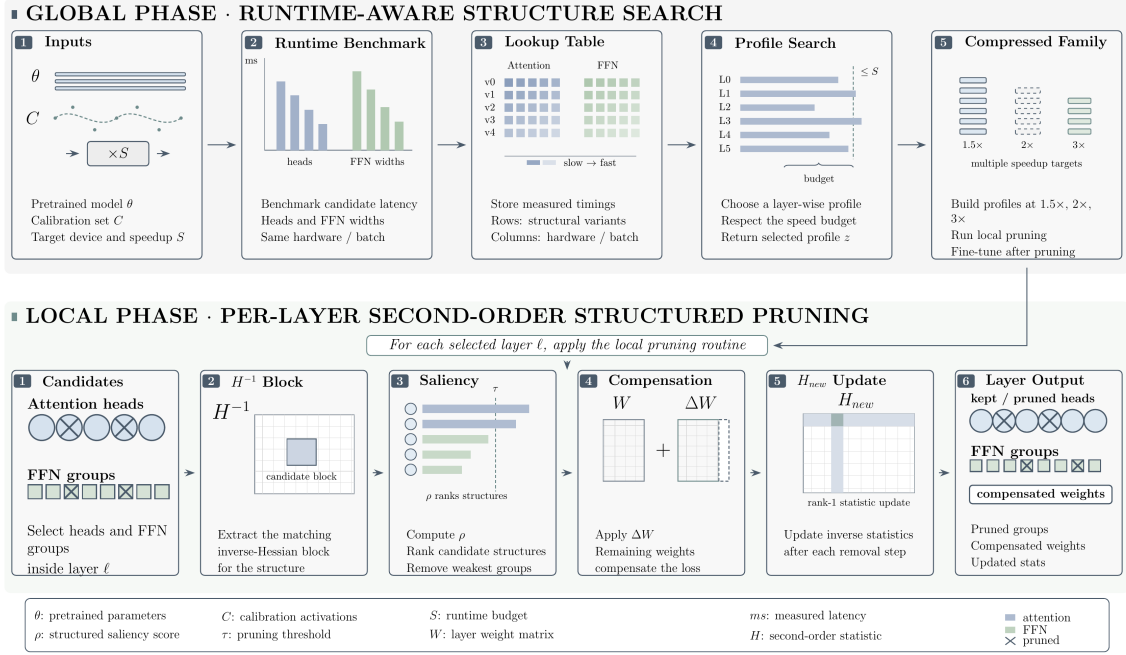


Figure 3.3: ZipLM pipeline. The global phase benchmarks attention-head and FFN configurations and searches for layer-wise sparsity profiles satisfying the required speedup. The local phase applies second-order structured pruning through inverse-Hessian scoring, compensation updates, and iterative Hessian refinement.

Second-order methods reduce local reconstruction error, but they do not fully resolve the allocation problem across the whole model. A method may know how to remove a head or channel with limited local damage and still not know how many heads or FFN channels should remain in each layer under a global budget. This is the point at which search-based methods become relevant.

3.2.5 Search-Based Methods

Search-based methods operate over complete sparsity configurations rather than applying a single closed-form ranking rule to individual units. This is especially relevant when the quality of a pruning decision depends on how several removals interact across layers.

Constrained Evolutionary Frameworks

EvoPress. EvoPress **evopress2024** searches over discrete layer-wise compression profiles instead of assigning a single saliency score per layer. A candidate is a configuration $\mathbf{C} = \{c_1, \dots, c_L\}$ where $c_\ell \in \mathcal{S}_\ell$ is the compression level for layer ℓ . Budget is preserved by level-switch mutation: one layer becomes more compressed only if another becomes less compressed, keeping the global average sparsity S_{glob} fixed. Mutations are deliberately local, with most offspring differing from the parent by only one or two switches.

Selection proceeds in multiple rounds with increasing token budgets and decreasing survivor counts:

$$\mathcal{D}^{(r+1)} = \text{Top}_{n_{r+1}} \left(\left\{ \mathcal{F}_{T_r}(C) \mid C \in \mathcal{D}^{(r)} \right\} \right), \quad T_1 < T_2 < \dots, n_1 > n_2 > \dots, \quad (3.16)$$

where the fitness signal is KL divergence to the reference model. This progressive evaluation scheme is what makes EvoPress practically efficient: early rounds use few tokens to discard poor candidates quickly, reserving the full calibration budget for finalists. The method converges in approximately 5–6 GPU hours for a standard Llama-2-7B sparsity search.

Figure ?? summarizes this search loop. The important point is that the algorithm evaluates complete compression profiles, selects promising candidates with increasing evaluation budgets, and mutates the retained profile while preserving the global compression constraint.

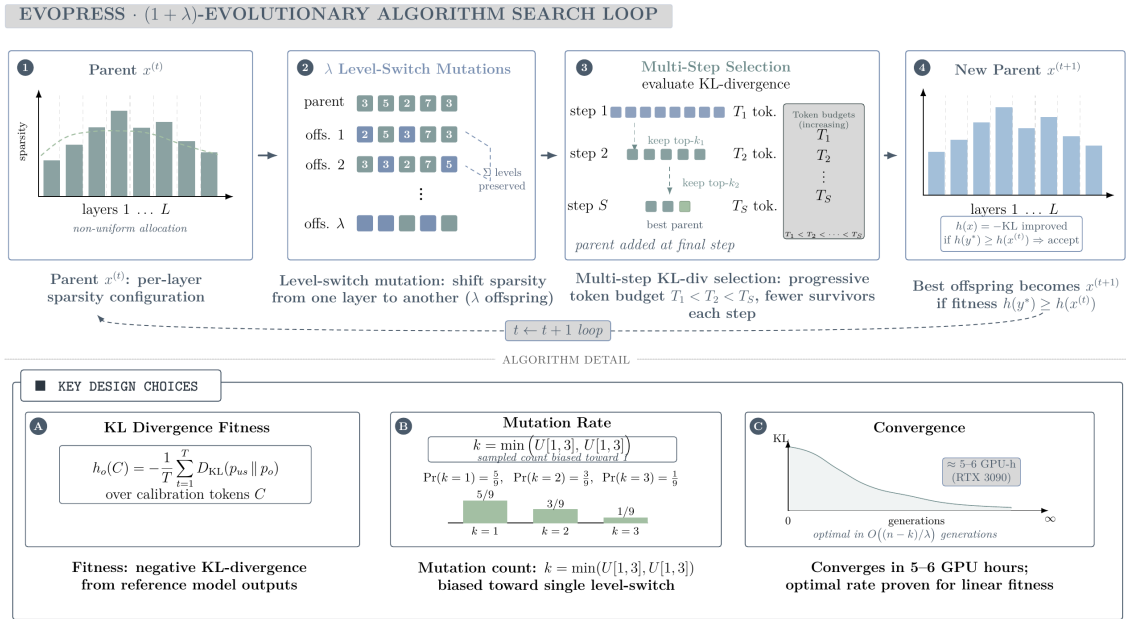


Figure 3.4: EvoPress $(1 + \lambda)$ evolutionary search loop for dynamic LLM compression (evopress2024). The method applies budget-preserving level-switch mutation and progressively evaluates candidates with larger calibration budgets.

DarwinLM. DarwinLM darwinlm2025 extends evolutionary search to *structured* pruning by unifying OBS-style second-order pruning with a training-aware offspring selection strategy. It differs from EvoPress on three axes: structured granularity (heads and FFN channels), front-loaded Hessian computation stored in a sparsity-level database (SLD), and fitness measured after a short lightweight finetune rather than from the one-shot pruned model.

Sparsity Level Database

For each module, DarwinLM solves the structured OBS problem at N_l equally-spaced sparsity levels and stores the pruned-and-compensated weights. The optimal structured mask and compensation are:

$$M^* = \arg \min_M \sum_{i=1}^{d_{\text{row}}} \mathbf{W}_{i,M} (\hat{H}_{MM}^{-1})^{-1} \mathbf{W}_{i,M}^\top, \quad (3.17)$$

$$\delta^* = -\mathbf{W}_{:,M} (\hat{H}_{MM}^{-1})^{-1} \hat{H}_{M,:}^{-1}, \quad (3.18)$$

where d_{row} is the output dimension of the module weight matrix, M indexes the pruned columns, and $\hat{H}$ is the empirical Hessian computed from calibration activations. During search, candidate models are assembled by looking up stored entries, avoiding any Hessian recomputation.

Training-Aware Selection

Each candidate receives a short lightweight finetune before evaluation:

$$\mathcal{F}(C_k) = D_{\text{KL}}(\hat{M} \parallel \text{finetune}(C_k, T_f)), \quad (3.19)$$

where $\hat{M}$ is the output distribution of the dense reference model and T_f is the fine-tuning token budget. Selection uses three progressive rounds ($T_f \in \{10\text{K}, 50\text{K}, 200\text{K}\}$ tokens) with decreasing survivor counts. The SLD must be computed once per model and sparsity range, making the upfront cost proportional to $L \cdot N_l$ structured OBS operations. DarwinLM compresses Llama-2-7B to 2.7B parameters with higher downstream accuracy than ShearedLlama using $5\times$ fewer post-training tokens (10B vs. 50B).

Algorithm Overview

DarwinLM sits between a post-training and an iterative regime: the SLD is built offline without end-to-end training, but each search candidate receives a lightweight fine-tune before fitness evaluation. In the offline phase, the SLD is built: for each module and each of N_l sparsity levels, the structured OBS problem is solved and the pruned-and-compensated weights are stored, so that any candidate configuration can be assembled by table lookup rather than recomputation. In the search phase, candidate models are stitched together by selecting one SLD entry per module, then evaluated through that lightweight finetune. Mutation follows the same budget-preserving logic as EvoPress, swapping one module’s sparsity level up only if another’s is swapped down. Selection proceeds over three rounds with token budgets of 10K, 50K, and 200K and shrinking survivor counts, so weaker candidates are eliminated cheaply before the full evaluation budget is spent. The upfront SLD cost is proportional to $L \cdot N_l$ structured OBS operations and is paid once per model.

Figure ?? shows the corresponding DarwinLM workflow. Compared with EvoPress, the key visual distinction is the front-loaded sparsity-level database: candidate models are assembled from stored structured-pruning choices, then filtered through training-aware selection.

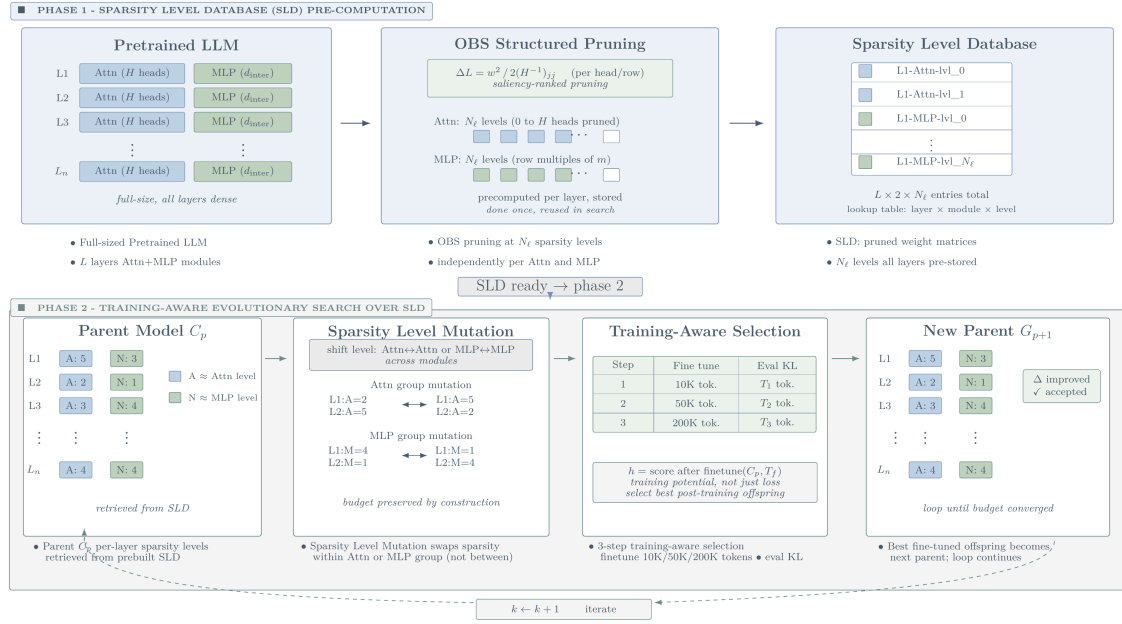


Figure 3.5: DarwinLM search loop (darwinlm2025). The method first builds a sparsity-level database with OBS-style structured pruning, then performs training-aware evolutionary selection over stitched candidate models.

Table ?? compares the practical preparation cost of the reviewed families. This table is included separately from the empirical results because a method can report strong accuracy while still requiring a substantially different calibration, recovery, benchmarking, or search budget.

Table 3.2: Preparation cost and evaluation requirements of representative pruning methods. The table separates low-cost post-training scoring, training-dependent structured pruning, curvature-based reconstruction, and population-based search.

Method	Dominant preprocessing cost	Additional requirement
Magnitude	Linear scan $O(W)$	None
Wanda	Activation collection and linear scoring	Small calibration set (≈ 128 samples)
CoFi	Multi-epoch joint mask and weight optimization	Task-specific teacher; ≈ 33 h for MNLI
LLM-Pruner	Forward/backward scoring over dependency groups	LoRA recovery (≈ 2.5 h on RTX 4090)
SparseGPT	Block-wise Hessian estimation and reconstruction	Calibration Hessian
MRP	Group-wise mask search and compensation	Block curvature updates for N:M groups
Fast PTPF	Diagonal or block Fisher estimation with mask tuning	Calibration set; optional latency lookup table
ZipLM	Layer-wise Hessian updates and runtime-table search	Calibration set plus hardware benchmarking pass
EvoPress / DarwinLM	Population search over compression profiles	1–6 GPU-hours; SLD pre-computation for DarwinLM

After the main method families have been described, the next step is to compare what they actually report. The empirical results below are therefore organized by pruning regime rather than by derivation, because perplexity, task accuracy, speedup, and recovery cost are not interchangeable measurements.

3.2.6 Empirical Comparison

The empirical evidence is organized by pruning regime. This avoids treating incompatible measurements as if they were directly comparable: unstructured and N:M methods mainly report perplexity under weight sparsity, structured methods report task accuracy or speedup after architectural removal, and search-based methods report the quality of full pruning configurations under budget constraints.

Unstructured and Semi-Structured Results

Table ?? keeps representative low, medium, and large model settings rather than listing every available row. The unstructured rows compare magnitude pruning, SparseGPT, and Wanda at 50% sparsity, while the 2:4 rows show the semi-structured regime that motivates group-aware compensation such as MRP.

Table 3.3: Representative unstructured and semi-structured pruning results on WikiText perplexity. PPL denotes perplexity; lower is better.

Model	Setting	Dense PPL	Magnitude PPL	SparseGPT PPL	Wanda PPL
LLaMA-7B	50% unstructured	5.68	17.29	7.22	7.26
LLaMA-30B	50% unstructured	4.77	7.54	5.31	5.24
LLaMA-65B	50% unstructured	3.56	5.90	4.57	4.57
OPT-13B	50% unstructured	10.13	1.2×10^4	11.17	–
OPT-175B	50% unstructured	8.35	4.3×10^4	8.21	–
OPT-13B	2:4 semi-structured	10.13	–	12.96	–
OPT-66B	2:4 semi-structured	9.34	–	10.09	–
OPT-175B	2:4 semi-structured	8.35	–	8.74	–

The table shows two separate points. First, pure magnitude pruning can fail badly at 50% sparsity, while activation-aware and second-order methods preserve perplexity much more effectively. Second, semi-structured 2:4 pruning remains more constrained than unconstrained unstructured pruning, so the compensation mechanism must respect the local group structure.

Structured Pruning Results

Table ?? summarizes representative structured-pruning results without listing every benchmark. The rows are selected to show low, medium, and high compression or speedup regimes where available, while keeping the evaluation suite explicit: CoFi is reported on GLUE tasks and SQuADv1, Fast PTPF reports GLUE/SQuAD results, and ZipLM reports GLUE-style classification tasks as well as SQuADv1.

Chapter 3. Related Work

Table 3.4: Representative structured Transformer pruning results across recovery regimes. GLUE denotes the General Language Understanding Evaluation benchmark, SQuADv1 denotes Stanford Question Answering Dataset version 1.1, acc. denotes accuracy, and F1 denotes question-answering F1 score.

Method	Model	Setting	Key result
LLM-Pruner ma2023llmpruner	LLaMA-7B	20% prune ratio	98.6% zero-shot accuracy retention after LoRA recovery
	LLaMA-7B	50% prune ratio	83.6% zero-shot accuracy retention after LoRA recovery
CoFi xia2022structured	BERT Base	GLUE/MNLI, $\approx 12\times$ speedup	80.6 acc. versus 78.8 for TinyBERT ₄ at similar speedup
	BERT Base	SQuADv1, $\approx 8.7\times$ speedup	82.6 F1, matching TinyBERT ₄ at comparable speedup
Fast PTPF kwon2022fast	BERT Base	GLUE and SQuADv1, FLOPs-constrained pruning	60–70% FLOPs with $\leq 1\%$ accuracy/F1 drop
	BERT Base	GLUE and SQuADv1, latency-constrained pruning on V100	$1.34\times$ – $1.47\times$ geometric-mean speedup depending on batch size
ZipLM kurtic2023ziplm	BERT Base	GLUE/QNLI and MNLI, $2\times$ speedup	91.4 / 84.8 acc.; essentially dense-level performance
	BERT Base	GLUE/QNLI and MNLI, $10\times$ speedup	88.6 / 82.7 acc.; approximately 5M parameters
	BERT Large	SQuADv1, $10\times$ speedup	89.1 F1 with 20.9M parameters

The structured results are closer to deployment because they remove executable units, but they are also harder to compare directly. GLUE classification tasks and SQuADv1 question answering measure different behavior, and the recovery protocols also differ: CoFi uses task-specific training, LLM-Pruner depends on LoRA recovery, Fast PTPF is largely post-training, and ZipLM adds device-specific runtime profiling.

Evolutionary and Search-Based Results

Table ?? keeps representative search-based results and includes both EvoPress and DarwinLM. EvoPress illustrates configuration search for depth pruning and sparsity allocation, while DarwinLM shows structured evolutionary pruning with training-aware offspring selection.

Table 3.5: Representative search-based pruning results. Wiki2 and C4 are perplexity values where lower is better; task average is accuracy where higher is better.

Setting	Model	Budget	Method	Wiki2↓	C4↓	Task Avg.↑
Depth pruning	Mistral-7B	25%	EvoPress	8.66	12.04	–
	Mistral-7B	25%	ShortGPT	43.26	40.16	–
	Mistral-7B	25%	Shortened LLaMA	14.94	19.30	–
Depth pruning	Llama-2-7B	37.5%	EvoPress	17.98	18.91	–
	Llama-2-7B	37.5%	ShortGPT	70.94	63.51	–
	Llama-2-7B	37.5%	Shortened LLaMA	35.37	26.07	–
Unstructured allocation	Mistral-7B	70%	EvoPress	14.42	16.46	53.8
	Mistral-7B	70%	OWL	17.22	21.66	51.9
	Mistral-7B	70%	Uniform	23.08	30.03	49.9
Structured search	Llama-2-7B	2.7B model	DarwinLM	–	–	62.8 after 10B-token post-training
	Llama-2-7B	2.7B model	ShearedLLaMA	–	–	62.6 after 50B-token post-training

The search results support the same conclusion from two directions. EvoPress improves over local or heuristic allocation when multiple removals interact, and DarwinLM shows that training-aware evolutionary selection can produce stronger structured models with less post-training data than a fixed pruning baseline.

Cross-Regime Synthesis

Table ?? aggregates the evidence at a higher level. It is not meant to replace the detailed result tables above; it summarizes what each regime establishes for the design of a pruning-search method.

Table 3.6: Aggregated interpretation of the empirical pruning evidence by regime.

Regime	Representative setting	Selected evidence	Reading
Weight-level unstructured	LLaMA-7B, 50% sparsity	Dense PPL 5.68; magnitude 17.29; SparseGPT 7.22; Wanda 7.26.	Better saliency preserves perplexity, but the resulting pattern remains unstructured and does not guarantee dense-kernel speedup.
Semi-structured N:M	OPT-13B, 2:4 sparsity	Dense PPL 10.13; SparseGPT-style compensation gives 12.96 under the 2:4 constraint.	N:M sparsity needs group-aware compensation, which motivates MRP-style joint treatment of simultaneously removed weights.
Structured LLM pruning	LLM-Pruner on LLaMA-7B	20% prune ratio retains 98.6% zero-shot accuracy after LoRA recovery; 50% retains 83.6%.	Dependency-aware pruning keeps the model executable, but the final quality depends strongly on recovery.
Structured encoder pruning	CoFi, Fast PTPF, and ZipLM on BERT	CoFi reaches 80.6 MNLI acc. near 12× speedup; Fast PTPF keeps 60–70% FLOPs with ≤1% drop; ZipLM reaches 88.6/82.7 QNLI/MNLI acc. at 10× speedup.	Structured pruning can produce executable acceleration, but the protocols differ in training cost, hardware profiling, and recovery budget.
Search-based pruning	EvoPress and DarwinLM	EvoPress improves depth-pruning and sparsity-allocation baselines; DarwinLM reports 62.8 average accuracy after 10B-token post-training versus 62.6 for ShearedLLaMA after 50B tokens.	Configuration search addresses global allocation, while training-aware selection improves structured candidates beyond one-shot scoring.

3.3 Discussion

The previous sections described each method family on its own terms. Read together, they answer two separate questions. The first is how to score a single removable unit, which is where magnitude, gradient, and curvature criteria disagree. The second is how to spread a fixed budget across the model, which is where the families disagree far more. Most of the unresolved arguments in the literature concern the second question, so the discussion takes it up first and then turns to what the empirical evidence shows.

3.3.1 Local Scoring Versus Global Allocation

The methods reviewed in this chapter differ in implementation, pruning unit, and recovery cost, but they can all be related to the same underlying objective:

$$\min_{\mathcal{M} \in \mathcal{F}} \Delta \mathcal{L}(\mathcal{M}), \quad (3.20)$$

where $\mathcal{M}$ denotes a feasible sparsity pattern or structured subnetwork and $\Delta \mathcal{L}$ denotes the loss increase caused by pruning. The difficulty is that $\mathcal{M}$ must satisfy a compression budget while preserving the behaviour of the model.

A recurring limitation of pruning criteria is that they evaluate removable units before the final compressed model is specified. SparseGPT estimates local second-order reconstruction costs for groups of weights **frantar2023sparsegpt**, while Wanda ranks weights using a product of weight magnitude and input activation norm **sun2023simple**. Such criteria are effective for identifying locally removable parameters, but they do not by themselves determine how a fixed compression budget should be distributed across layers **hoefer2021sparsity**; **cheng2024pruningsurvey**. Empirically, one-shot local pruning is effective at lower compression ratios, but iterative or globally-allocating methods prove superior at higher pruning rates **janusz2025oneshot**. This gap is recognised by recent allocation-based methods: ZipLM explicitly models both local correlations within layers and global correlations across layers, and EvoPress searches over layer-wise compression profiles to minimise output distribution divergence **kurtic2023ziplm**; **evopress2024**.

3.3.2 Limitations of Local Ranking at High Compression

The preceding discussion identifies a consistent pattern across the reviewed method families. Local ranking criteria can be effective at low-to-moderate compression, but their limitations become more visible as the target compression ratio increases **hoefer2021sparsity**; **sanh2020movement**; **bai2024sparsellm**; **janusz2025oneshot**. This behaviour follows from the structure of the pruning problem: local criteria score removable units before the final compressed model is fully specified.

For Transformer models, structured pruning is naturally expressed as a layer-wise allocation problem over executable units. A pruned Transformer may retain different numbers of attention heads, FFN intermediate dimensions, hidden dimensions, or even complete MHA/FFN modules in different layers **xia2022structured**; **kurtic2023ziplm**; **cheng2024pruningsurvey**. Thus, for a model with L layers, H attention heads per layer, and F FFN channels per layer, the retained structure can be represented by an allocation vector

$$\mathbf{s} = (h_1, f_1, h_2, f_2, \dots, h_L, f_L),$$

subject to a global MAC budget Ω . Recent layer-wise pruning work therefore treats compression as the problem of assigning adaptive sparsity ratios across layers rather than applying a single fixed pruning rule everywhere **li2024dsa**.

A local ranking method instead assigns a saliency score to each removable unit and removes the lowest-scoring units until the budget is met. This implicitly approximates the global alloca-

tion problem by assuming that the contribution of each removal can be evaluated independently. EvoPress identifies this assumption explicitly in depth pruning, where block-scoring methods assume error linearity: each block is treated as contributing independently to the total compression error **evopress2024**. ZipLM motivates its structured pruning algorithm from the opposite direction, arguing that effective pruning must account for both local correlations within layers and global correlations across layers **kurtic2023ziplm**.

This independence approximation is violated by the compositional structure of Transformers. Each Transformer block contains residual-connected attention and feed-forward sublayers, and self-attention layers operate on representations produced by previous layers **vaswani2017attention**. In pruning terms, this means that modifying one layer changes the input distribution seen by later layers. SparseLLM formalises this issue by treating LLMs as chains of modules whose outputs feed subsequent modules, and argues that minimising only local layer error leads to suboptimal global pruning, especially in high-sparsity regimes **bai2024sparsellm**.

The same coupling appears in attention and in the global budget constraint. Scaled dot-product attention normalises query-key scores with a softmax, while multi-head attention combines several heads operating on the same layer input **vaswani2017attention**. Empirically, attention heads have heterogeneous importance: many heads can be removed, but excessive head removal causes catastrophic drops, and sensitivity differs across attention components **micheel2022not**. Moreover, under a fixed MAC budget, retaining more capacity in one layer requires removing capacity elsewhere. Prior work shows that non-uniform sparsity across layers is important at high compression, and DSA directly addresses this by searching for adaptive layer-wise sparsity allocation functions **gale2019state**; **sanh2020movement**; **li2024dsa**.

3.4 Empirical Results

Having argued that allocation matters more than local scoring at high compression, the next question is whether the reported numbers bear that out. They largely do, but only once methods that measure different things are kept apart. The unstructured results isolate the saliency criterion under weight sparsity; the structured and search-based readings then bring recovery cost and budget allocation back into the picture.

3.4.1 Unstructured and Semi-Structured Methods

The unstructured pruning results allow three clean comparisons across increasing levels of local sophistication: magnitude baseline, Wanda (activation-aware saliency), and SparseGPT (second-order compensation).

The SparseGPT results primarily highlight the instability of pure magnitude pruning at high sparsity. At 50% sparsity on OPT-13B, magnitude pruning produces a perplexity of 1.2×10^4 compared to the dense model’s 10.13. This indicates severe representational degradation under weight selection based solely on individual magnitude. SparseGPT reduces the same setting to 11.17, corresponding to a +1.04 increase over the dense model.

The comparison between Wanda and SparseGPT on LLaMA models is also instructive. At

50% sparsity on LLaMA-7B, Wanda achieves 7.26 versus SparseGPT’s 7.22, a negligible 0.04 point difference. On LLaMA-13B, Wanda actually outperforms SparseGPT by 0.06 points (6.15 versus 6.21). This apparent paradox reflects two facts. First, the Wanda activation norm $\|x_j\|_2$ is an effective proxy for the weight’s contribution to the output, particularly in models with large activation outliers, which LLaMA models exhibit. Second, the calibration set used for Hessian construction in SparseGPT introduces variance that at moderate sparsity levels can offset the theoretical advantage of curvature information. At higher sparsity levels (70%+), the second-order advantage of SparseGPT becomes more consistent, because local reconstruction errors accumulate and the precise compensation direction matters more.

3.4.2 Structured Methods

CoFi and ZipLM deliver similar accuracy at comparable speedup levels through substantially different preparation pipelines. CoFi reaches 80.6 on MNLI at approximately $12\times$ speedup after 33 hours of joint mask and weight optimisation. ZipLM reaches 81.7 at $12\times$ through a fine-tuning schedule interleaved with structured pruning passes and a dedicated hardware benchmarking stage. In both cases the reported accuracy reflects the full pipeline rather than the pruning criterion in isolation, which makes the two methods difficult to compare directly.

3.4.3 Cross-Method Comparison

The tables above support two conclusions. At 50% unstructured sparsity on OPT-13B, magnitude pruning collapses to a perplexity of 1.2×10^4 while SparseGPT reaches 11.17 (Table ??), showing that second-order compensation is necessary once individual weight interactions become non-negligible. At 70% unstructured sparsity on Mistral-7B, EvoPress reaches 14.42 perplexity while uniform allocation reaches 23.08 (Table ??), and at 37.5% depth sparsity on Llama-2-7B, EvoPress gives 17.98 against ShortGPT’s 70.94, showing that global allocation search becomes the dominant factor at high compression.

3.5 Limitations of the Reviewed Methods

The comparisons so far reveal real differences between methods, yet much of the reported gap reflects how each paper was evaluated rather than how well it prunes. Three problems recur and distort almost every cross-paper claim: the evaluation regimes are inconsistent, the recovery budgets are uneven, and the results depend on a calibration set whose effect is seldom checked.

3.5.1 Evaluation Protocol Heterogeneity

A major limitation of the pruning literature is the absence of a standardised evaluation protocol. Methods are reported across at least five distinct evaluation regimes: (1) language model perplexity at fixed nominal sparsity, (2) zero-shot task accuracy after pruning with or without recovery, (3) supervised fine-tuning accuracy after pruning and recovery, (4) measured wall-clock speedup or latency on specific hardware, and (5) FLOPs reduction relative to the dense model.

These five regimes are not directly commensurable. A method reporting 99% of dense accuracy at $2\times$ FLOPs reduction (Regime 5) addresses a different evaluation objective from a method reporting $1.5\times$ wall-clock speedup with $<1\%$ accuracy drop (Regime 4). FLOPs constitute a hardware-agnostic theoretical measure whose relation to runtime depends on memory bandwidth, batch size, and the accelerator execution model. Some sparse execution formats require hardware-specific support, whereas standard unstructured masks often fail to yield proportional wall-clock speedup on general dense kernels.

This protocol fragmentation makes direct cross-paper comparison highly unreliable. A result table that places Wanda, CoFi, LLM-Pruner, ZipLM, and EvoPress in the same rows implicitly treats them as solutions to the same problem. In practice, these methods target different budgets, recovery regimes, and deployment objectives. Aggregating them without qualification creates a misleading impression of a single Pareto frontier.

3.5.2 Recovery Budget Inconsistency

Closely related is the problem of inconsistent recovery budgets. Structured pruning tends to incur larger accuracy drops than unstructured pruning at equal sparsity, making the recovery stage especially consequential **lightweight**. The nominal distinction between “post-training pruning” and “pruning with fine-tuning” masks a broad range of recovery costs that substantially affect reported accuracy. Within the structured methods reviewed, Fisher post-training pruning uses zero weight updates with only mask tuning and a total time of approximately 39 seconds; LLM-Pruner uses LoRA recovery on 50K samples for approximately 2.5 hours with rank-8 adapters; ZipLM uses gradual full-model fine-tuning across 8–10 epochs per pruning step with full distillation; and CoFi uses joint mask and weight optimisation across 40 epochs with a task-specific teacher.

A controlled comparison should hold the recovery budget constant rather than the nominal pruning algorithm. A CoFi model trained for 40 epochs is not directly comparable to an LLM-Pruner model with 2.5 hours of LoRA recovery. Under a matched fine-tuning budget, the observed gap could narrow substantially. The literature rarely reports such controlled comparisons, so part of the reported accuracy difference reflects recovery investment rather than pruning quality alone.

3.5.3 Calibration Set Sensitivity

All post-training pruning methods depend on a calibration set to estimate saliency scores, Hessian surrogates, or Fisher information matrices. The choice of calibration set affects the pruned model in ways that are rarely systematically studied.

For language models, the standard practice is to use 128–2048 samples from a general text corpus such as C4 or WikiText. This is a relatively small sample of the model’s training distribution, and it may not adequately represent the activation statistics that determine saliency. LLM-Pruner uses only 10 short sequences for calibration, far fewer than SparseGPT’s 128 samples, which increases variance in the Taylor scoring and may explain some of the variability in its

zero-shot accuracy results across runs. More fundamentally, all methods that use a calibration set implicitly assume that the saliency ranking derived from calibration samples is generalisable to the full evaluation distribution. Recent work confirms that calibration data is critical to post-training pruning, especially at high sparsity, and that data more similar to the pre-training distribution yields better pruning outcomes [ji2025calibration](#); [bandari2024calibration](#).

3.6 Conclusion

The methods reviewed in this chapter differ in what they score, what they remove, and how much recovery they require, but they share a common limitation at high compression: each method decides locally, before the joint effect of all removals is known. Magnitude and activation-aware methods score individual weights. Gradient and second-order methods improve that score but still apply it one unit at a time. Even dependency-aware structured methods such as LLM-Pruner rank groups independently across layers. The empirical results make the consequence visible: at aggressive depth or sparsity budgets, local ranking degrades rapidly while search over complete configurations does not.

Chapter 4

Conclusion

The three chapters in this report build toward a single observation. Chapter ?? established the formal vocabulary: the Transformer, the efficiency metrics that give compression a concrete meaning, and the genetic search framework that motivates the proposed contribution. Chapter ?? defined the pruning problem precisely, surveyed the main criteria from magnitude to second-order approximations, and described the Transformer structures that can be removed without breaking the forward pass. Chapter ?? placed the leading methods in context and identified where each one stops short.

That stopping point is the same across nearly all reviewed methods. Local scoring, whether by weight magnitude, activation statistics, gradient, or curvature, assigns importance to each removable unit before the joint effect of all removals is known. This works at moderate compression. At high compression, the allocation of the budget across layers matters more than the precision of any individual score, and local methods have no mechanism for reasoning about the full configuration. EvoPress makes this concrete: at 37.5% depth sparsity on Llama-2-7B, methods that score blocks locally reach perplexities above 70, while a search over layer combinations stays at 17.98.

This gap between local scoring and global allocation is the problem the proposed framework is designed to address. The literature reviewed here does not resolve it, but it defines it precisely enough that a search-based approach becomes the natural response.

Appendices